中国人民大學 RENMIN UNIVERSITY OF CHINA

# 第2讲 深度学习

**余力 博士**

教授、博士生导师

大数据分析与智能决策实验室

buaayuli@ruc.edu.cn

# 目录

1. 深度学习概述

2. 从回归说起

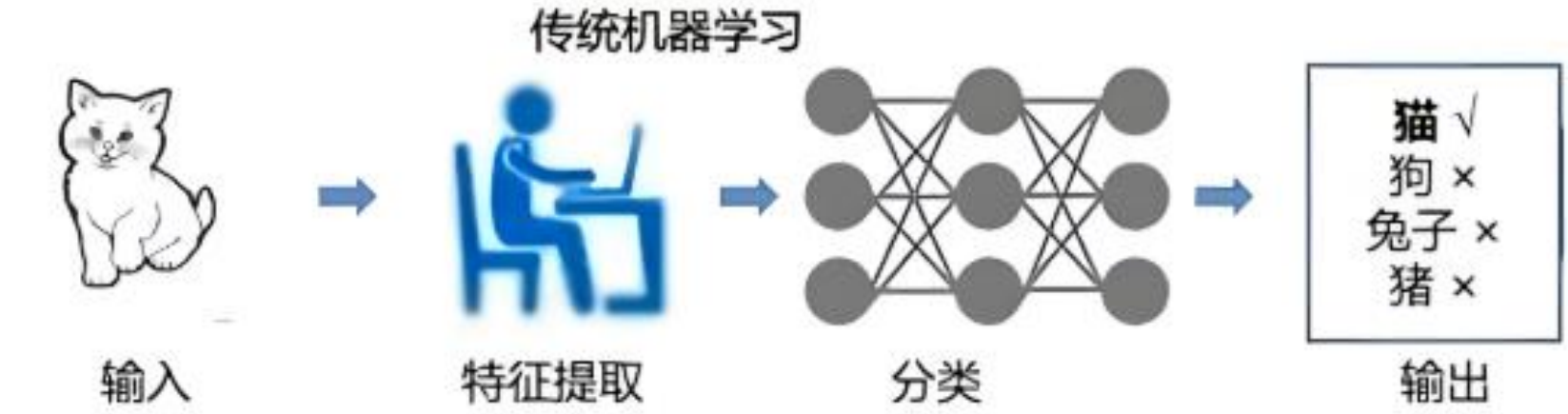
3. 神经网络

4. 经典深度学习



# 1. 深度学习概述

# 传统机器学习与深度学习






# 传统机器学习 VS 深度学习

## 深度学习时代，我们这么做...

步骤1：花几个星期时间收集并标注(框出狗屎位置)数万张便便图

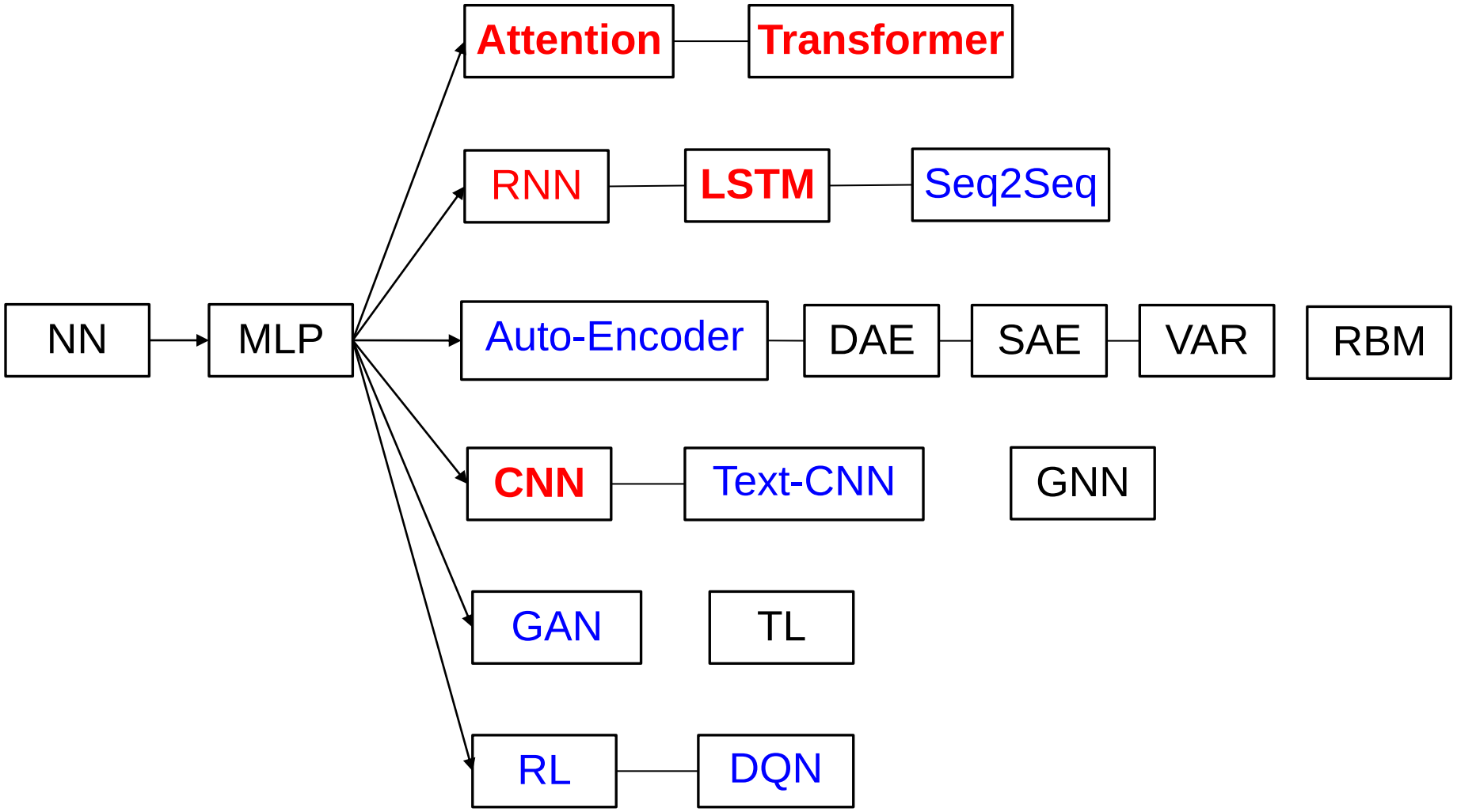
步骤2：花1个星期，挑几个深度模型，选几组模型超参数

步骤3：交给机器绞尽脑汁优化学模型中的数千万/数亿权重参数



+ **专家选择深度模型** + **机器优化深度模型**

# 深度学习技术概览


NN
MLP
Attention
Transformer
RNN
LSTM
Seq2Seq
Auto-Encoder
DAE
SAE
VAR
RBM
CNN
Text-CNN
GNN
GAN
TL
RL
DQN



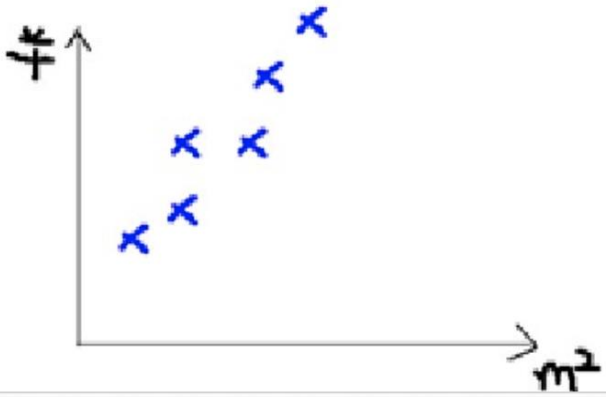
# 2. 从回归说起

# 一个简单的例子

## 问题引入

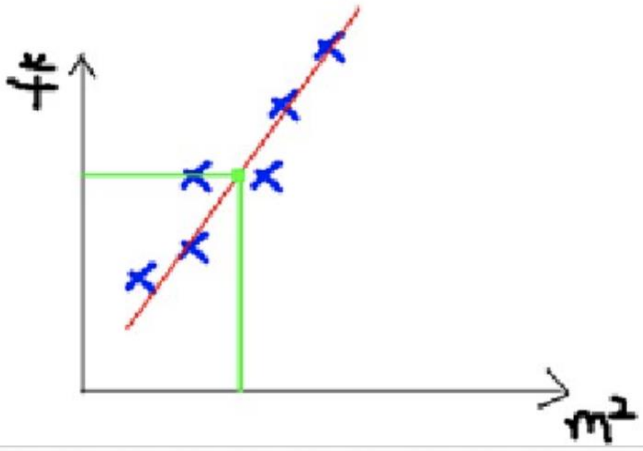
假设有一个房屋销售的数据如下：

| 面积(m^2) | 销售价钱（万元） |
|---|---|
| 123 | 250 |
| 150 | 320 |
| 87 | 160 |
| 102 | 220 |




如果来了一个新的面积，假设在销售价钱的记录中没有的，怎么处理？

解决方法：用一条曲线去尽量准的拟合这些数据，然后如果有新的输入过来，我们可以在将曲线上这个点对应的值返回。如果用一条直线去拟合，可能是下面的样子：



# 线性回归

- 线性回归：寻找X和Y之间的关系

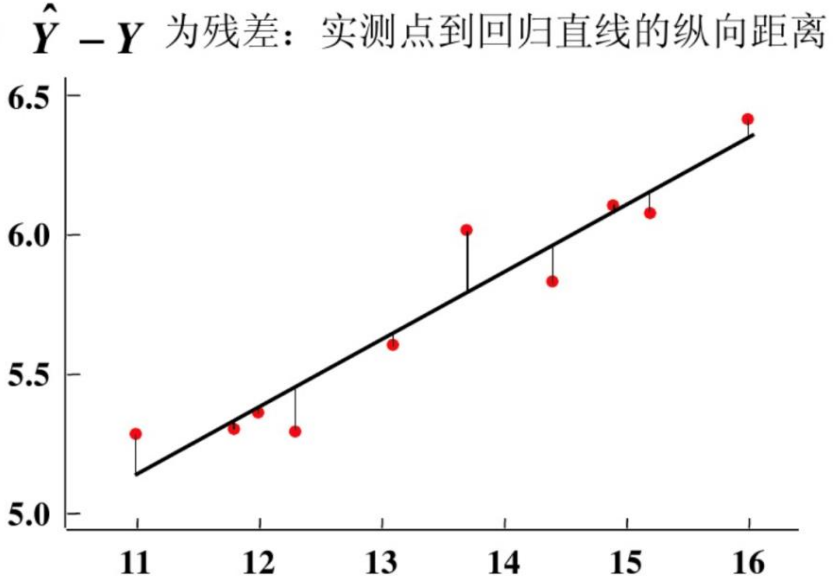
$$Y = \theta_0 + \theta_1 * x$$

理想的拟合直线

- 策略：误差平方和最小化准则：

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}\left(\hat{Y}^{(i)} - Y^{(i)}\right)^2$$

$$min_\theta \ J_\theta$$



# 最小二乘算法

$$= \frac{1}{2}\sum_{i=1}^{m}(\boldsymbol{\theta_0} + \boldsymbol{\theta_1 x_1^{(i)}} + \boldsymbol{\theta_2 x_2^{(i)}} - \boldsymbol{y^{(i)}})^2$$

$$J'_{\theta_0} = \sum_{i=1}^{\mathrm{m}} (\theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - \mathrm{y}^{(i)}) = 0$$

$$J'_{\theta_1} = \sum_{i=1}^{\mathrm{m}} x_1^{(i)}(\theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - \mathrm{y}^{(i)}) = 0$$

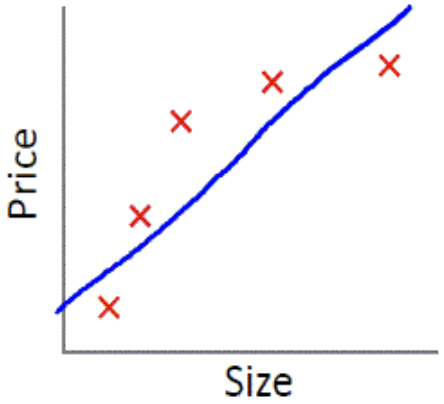
$$J'_{\theta_2} = \sum_{i=1}^{\mathrm{m}} x_2^{(i)}(\theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - \mathrm{y}^{(i)}) = 0$$

$$J(\mathrm{B}) = \frac{1}{2}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$$

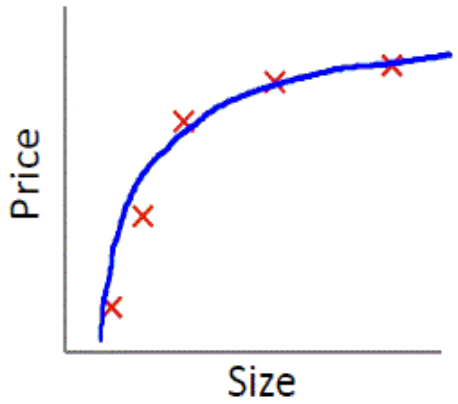
$$\hat{\theta} = (\mathrm{X}^T\mathrm{X})^{-1}X^TY$$

# 过拟合与欠拟合

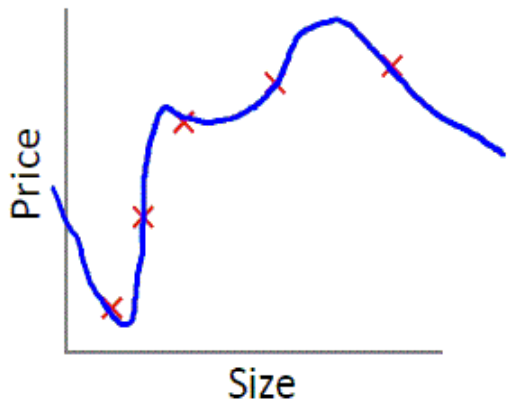
- **过拟合(overfitting)**：模型学的“太好”，把样本自身的一些特点当作所有潜在样本都会具有的一般性质，泛化性能很差。
- **欠拟合(underfitting)**：对训练样本的一般性质尚未学好。




$\theta_0 + \theta_1 x$




$\theta_0 + \theta_1 x + \theta_2 x^2$




$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

# 正则化

Ridge Regression
（岭回归）

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2}\sum_{\mathrm{j}}^{\mathrm{n}}\theta_{\mathrm{j}}^2$$

Lasso Regression
（套索回归）

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2}\sum_{\mathrm{j}}^{\mathrm{n}}|\theta_{\mathrm{j}}|$$

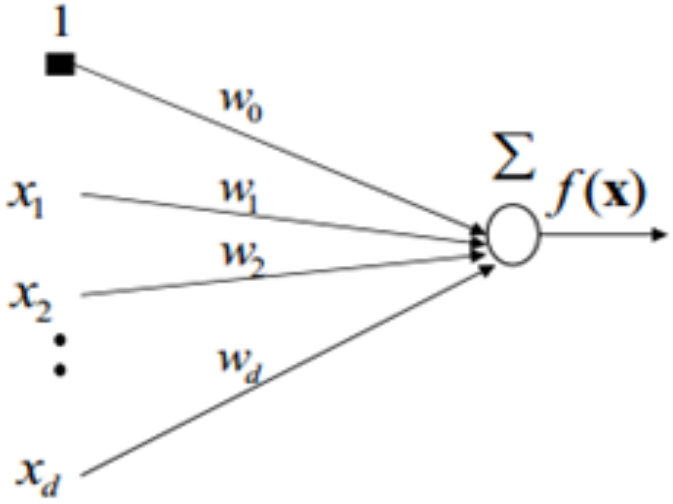
Elastic Net
（弹性网络）

$$\min_{w} \frac{1}{2n_{samples}}||Xw - y||_2^2 + \alpha\rho||w||_1 + \frac{\alpha(1-\rho)}{2}||w||_2^2$$

# 逻辑回归

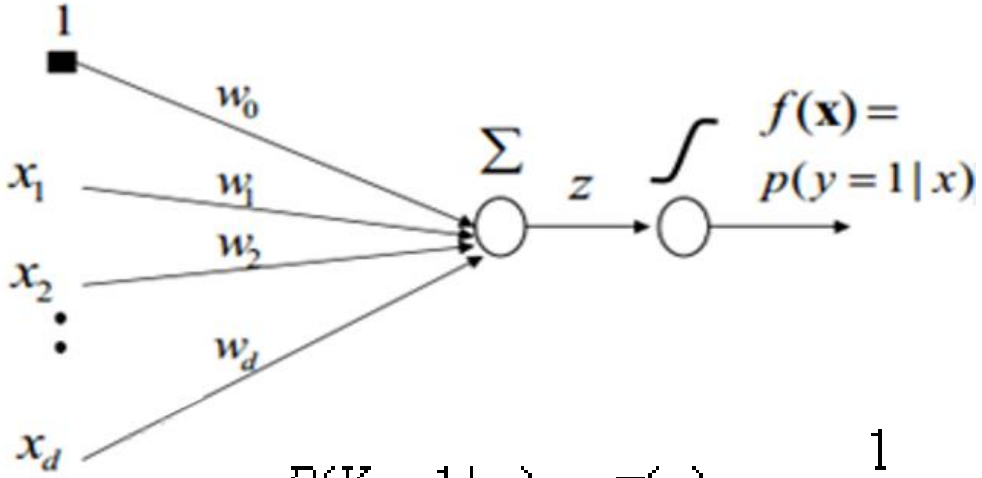
## Linear regression

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$$

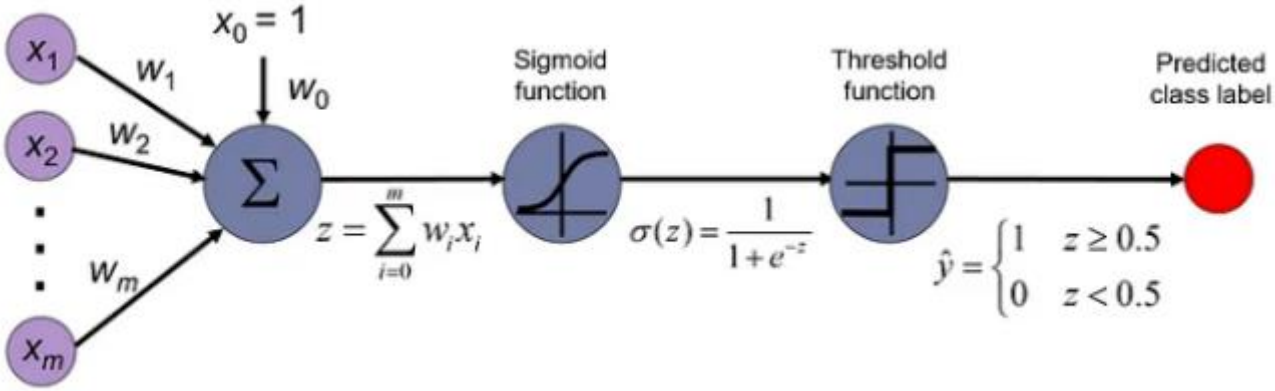



## Logistic regression

$$f(\mathbf{x}) = p(y=1 \mid \mathbf{x}, \mathbf{w}) = g(\mathbf{w}^T \mathbf{x})$$




$$P(Y=1 \mid x) = \pi(x) = \frac{1}{1+e^{-g(x)}}$$

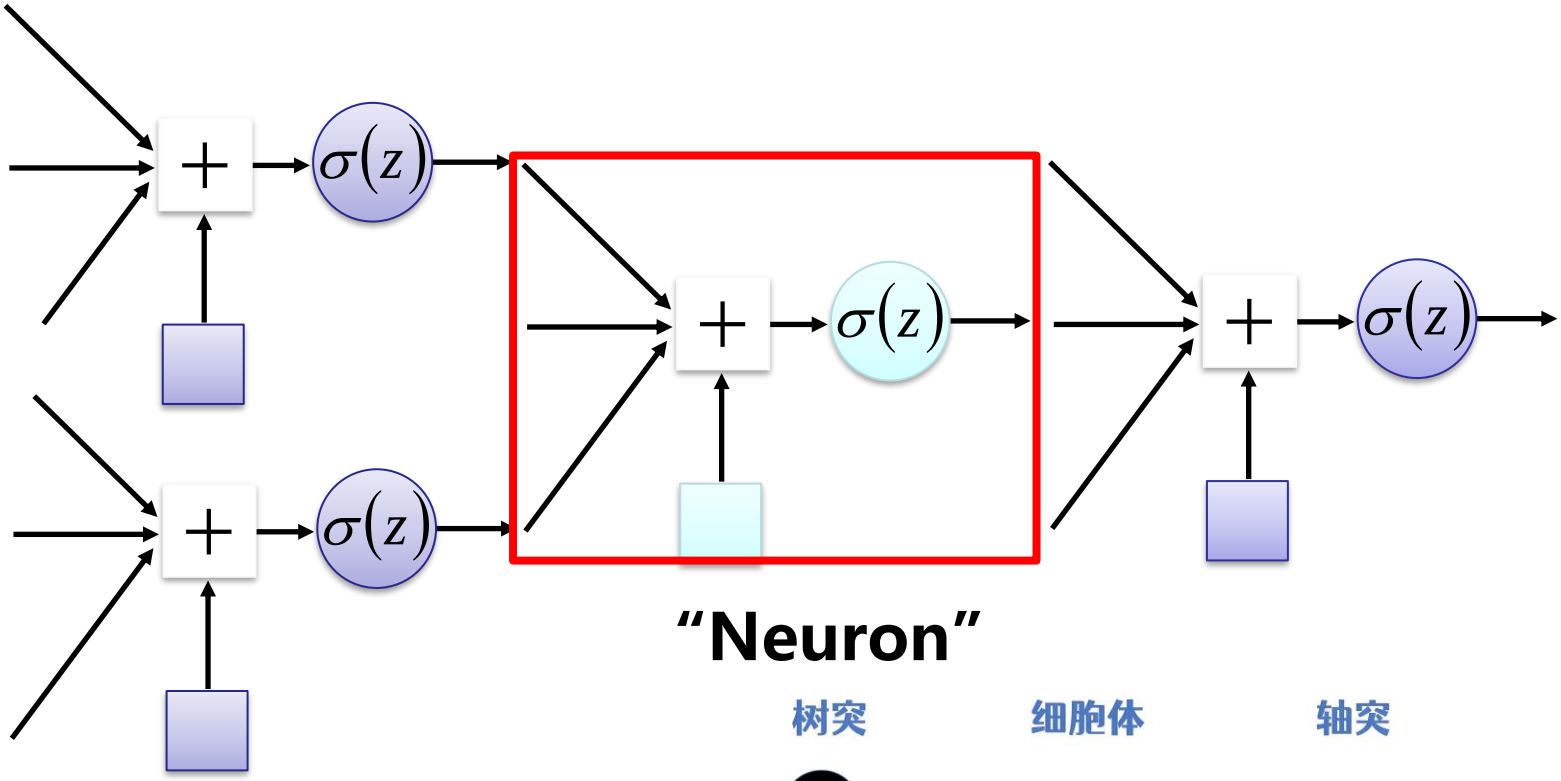



https://github.com/NLP-LOVE/ML-NLP/tree/master/Machine%20Learning/Liner%20Regression/demo

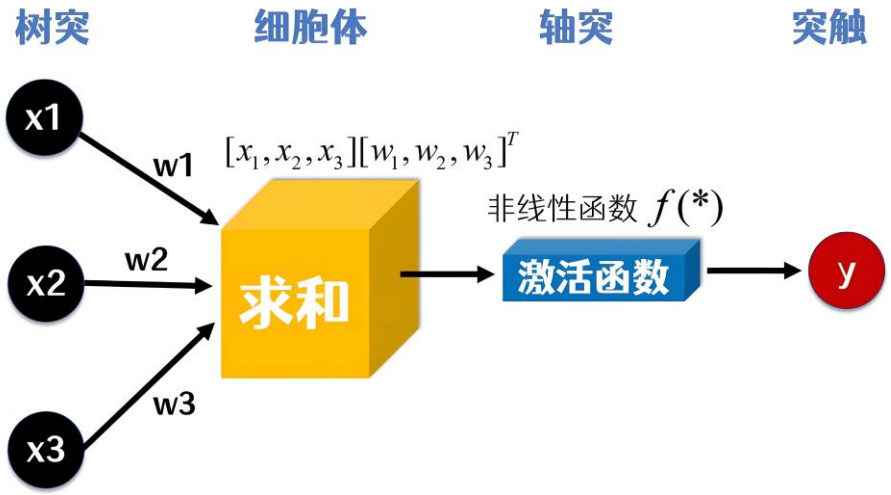


# 3. 神经网络

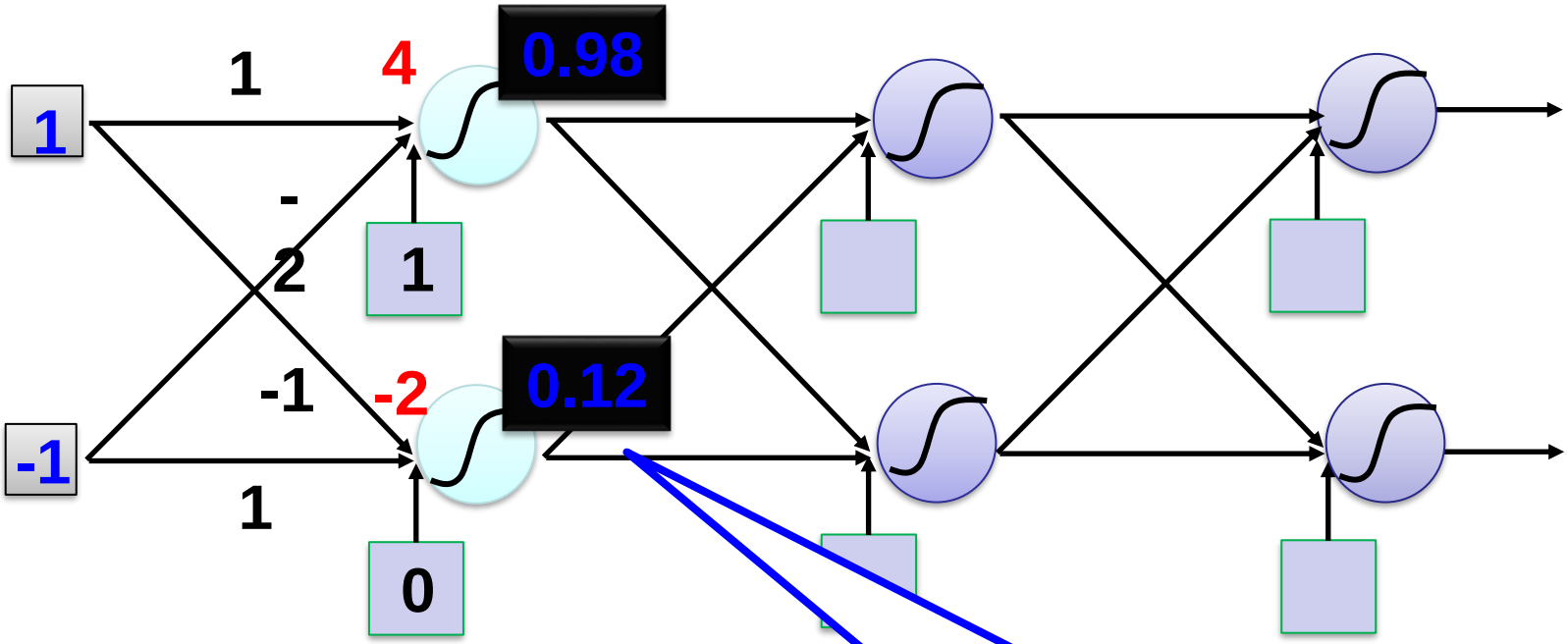
# Neural Network




**"Neuron"**



# Neural Network

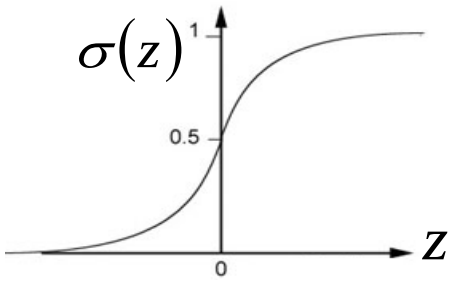



$$\sigma\left(\begin{bmatrix}1 & -2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\ -1\end{bmatrix}+\begin{bmatrix}1\\ 0\end{bmatrix}\right)=\begin{bmatrix}0.98\\ 0.12\end{bmatrix}$$

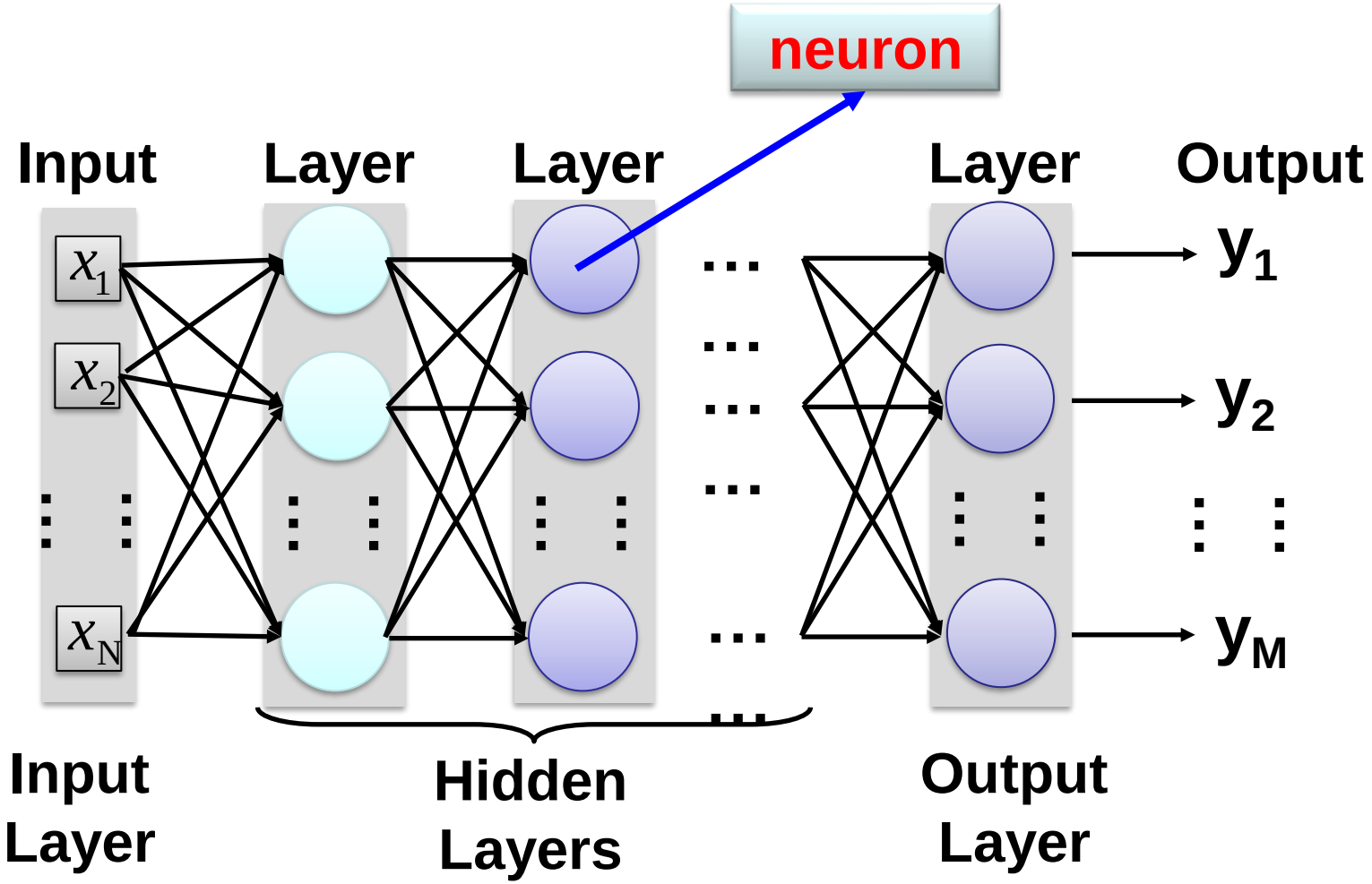
$$\begin{bmatrix}4\\ -2\end{bmatrix}$$

**Sigmoid Function**

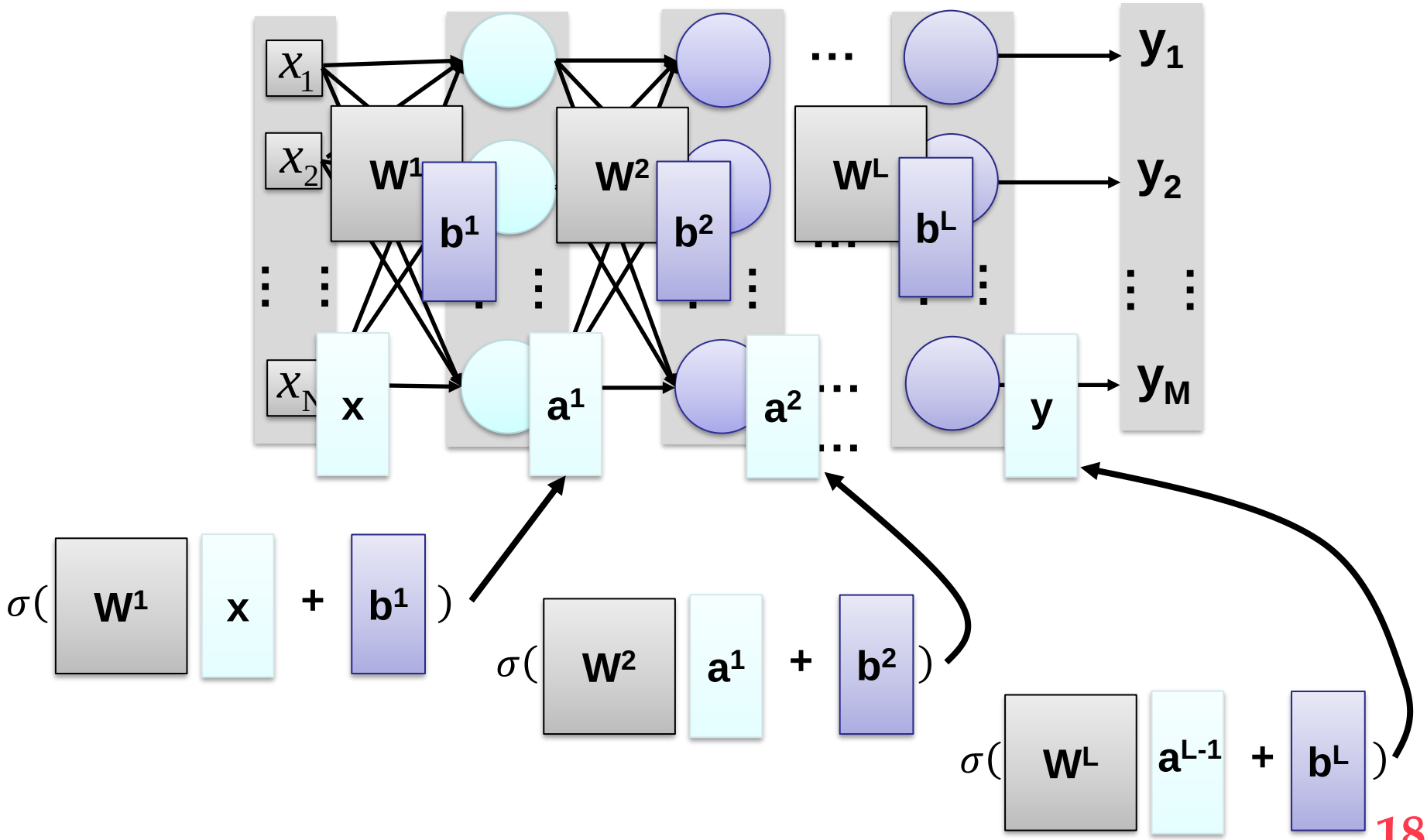
$$\sigma(z)=\frac{1}{1+e^{-z}}$$



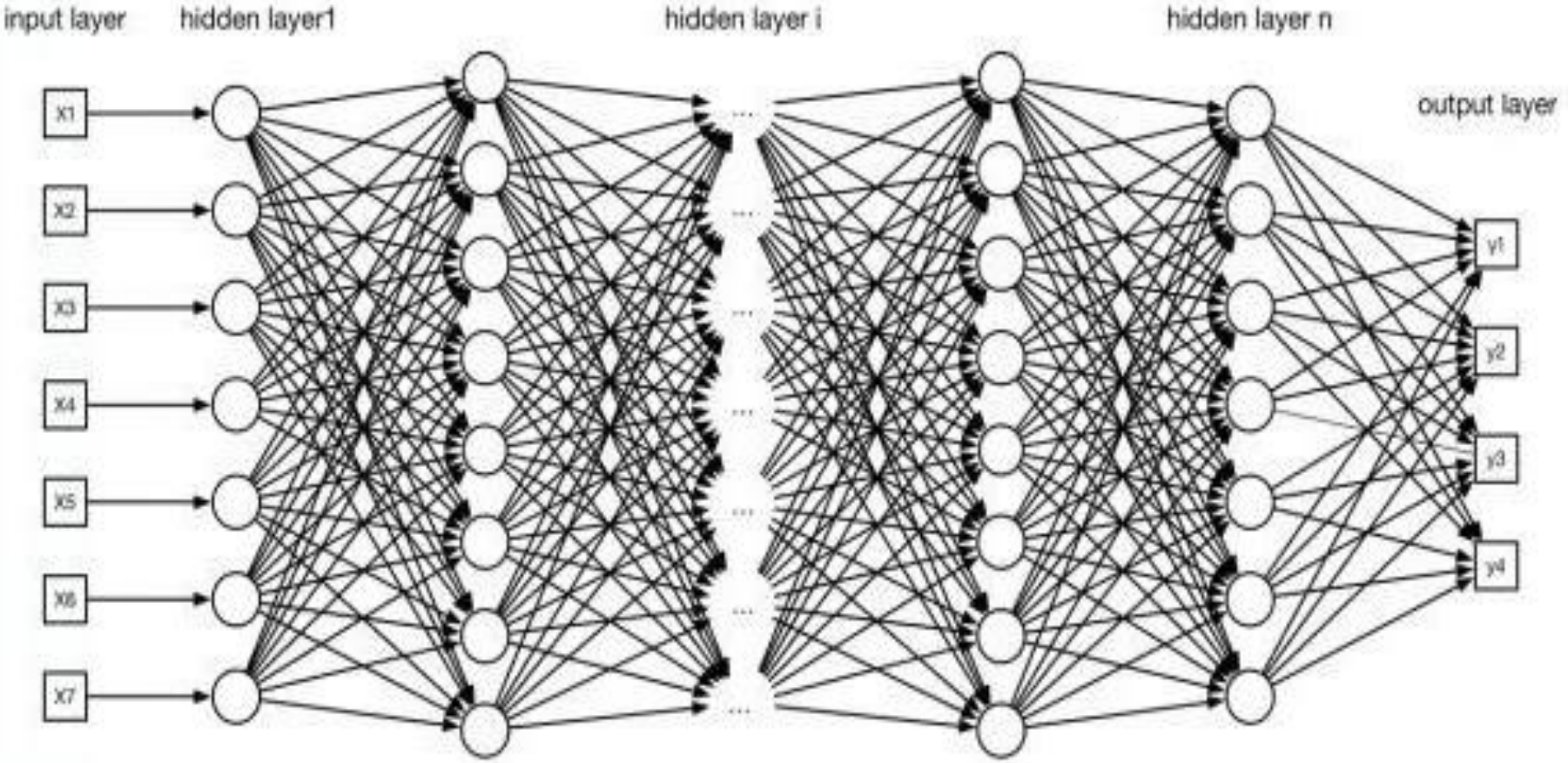
# Neural Network



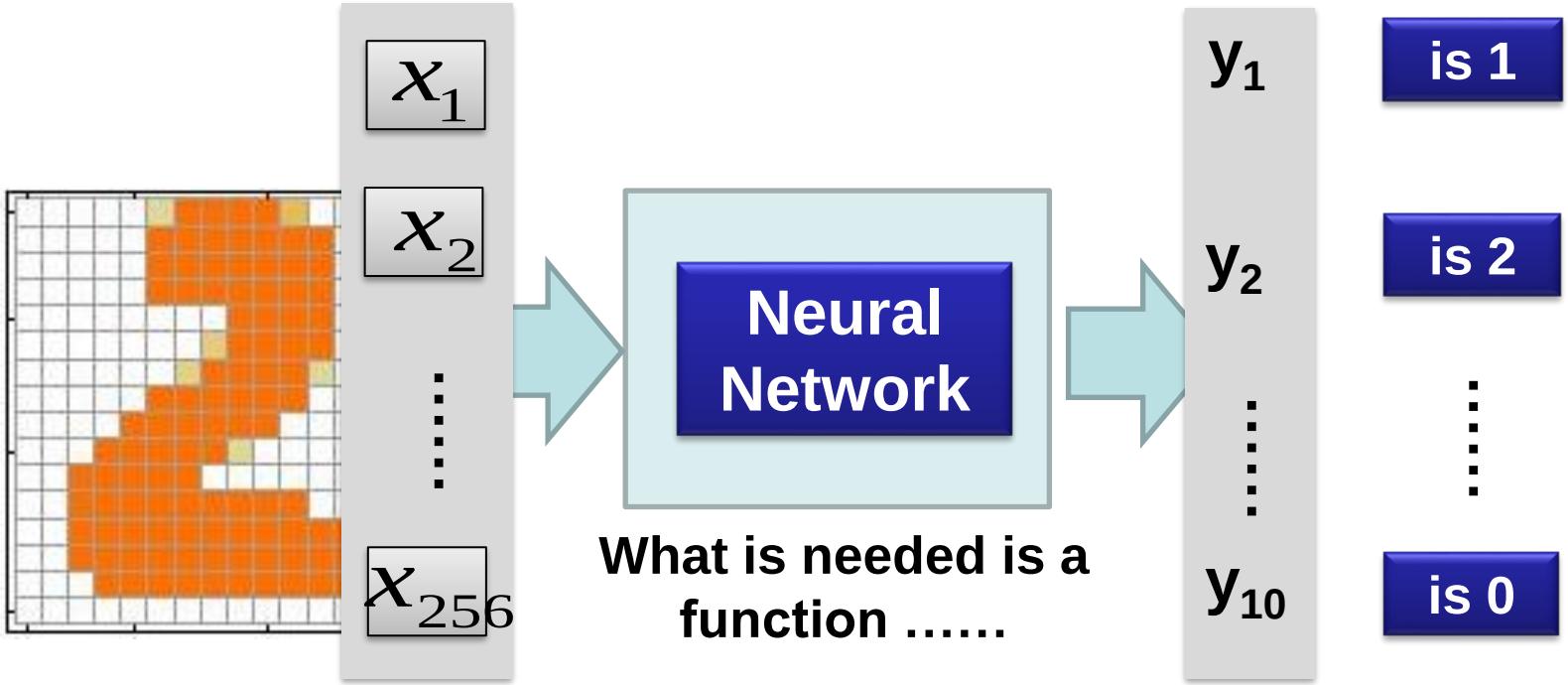
# Neural Network



# Multilayer Perceptron (MLP)



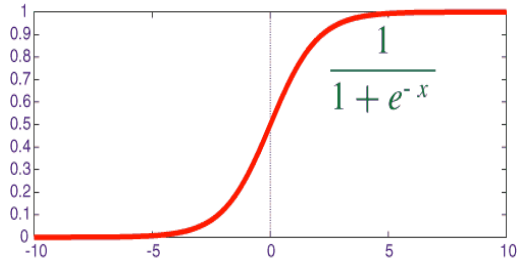
# Example Application



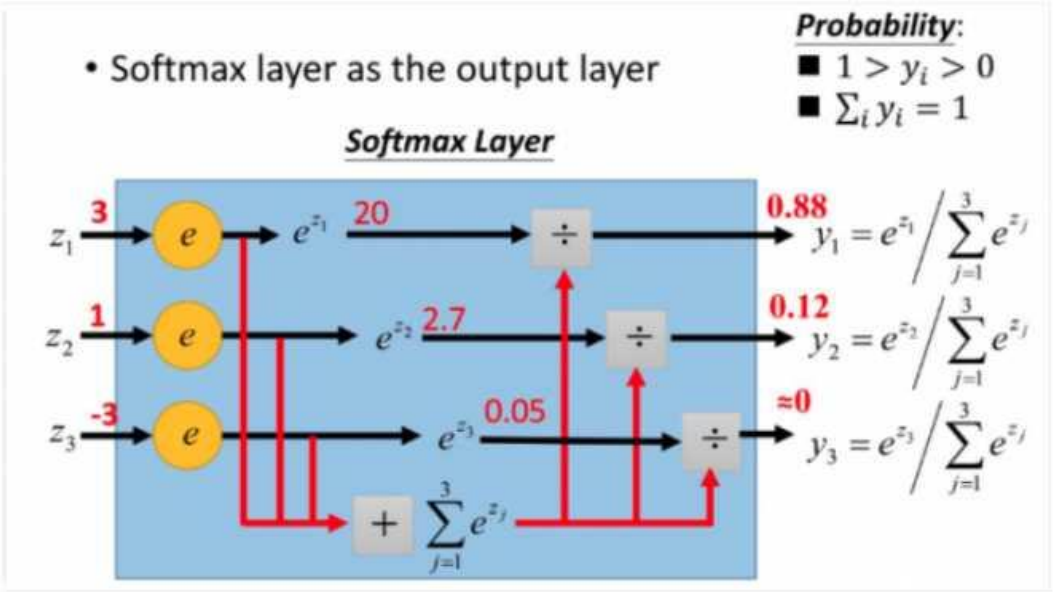
# 激活函数

## sigmoid

$$f(x) = \frac{1}{1+e^{-x}}$$

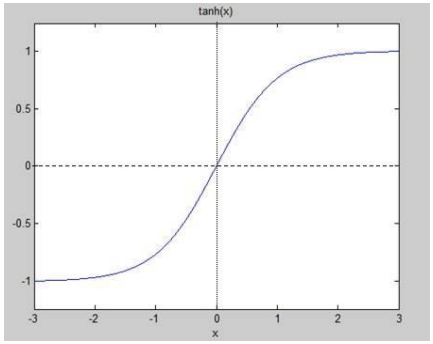



## softmax




## 正切函数（Tanh）

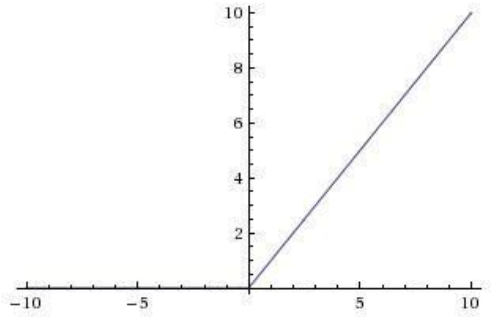
$$f(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$$




## 线性整流函数（ReLU）

$$f(x) = \begin{cases} x, & if x \geq 0 \\ 0, & if x < 0 \end{cases}$$

$$f(x) = max(0, x)$$



# 目标函数

MSE（Mean Squared Error）

**均方差**

$$loss = \frac{1}{2m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

**交叉熵**

$$H(p,q) = -\sum_{i=1}^{n} p(x_i)log(q(x_i))$$

**相对熵（KL散度）**

$$D_{KL}(p||q) = \sum_{i=1}^{n} p(x_i)log(\frac{p(x_i)}{q(x_i)})$$

$$loss = -\frac{1}{m}\sum_{j=1}^{m}\sum_{i=1}^{n} y_{ji}log(\hat{y_{ji}})$$

n为事件的所有可能性。

$D_{KL}$的值越小，表示q分布和p分布越接近

**多分类**

$$loss = -ylog(\hat{y}) - (1-y)log(1-\hat{y})$$

# 单分类与多分类

| * | 猫 | 青蛙 | 老鼠 |
|---|---|---|---|
| Label | 0 | 1 | 0 |
| Pred | 0.3 | 0.6 | 0.1 |

$$loss = -\frac{1}{m}\sum_{j=1}^{m}\sum_{i=1}^{n} y_{ji} log(\hat{y_{ji}})$$

$$loss = -ylog(\hat{y}) - (1-y)log(1-\hat{y})$$

| * | 猫 | 青蛙 | 老鼠 |
|---|---|---|---|
| Label | 0 | 1 | 1 |
| Pred | 0.1 | 0.7 | 0.8 |

$$loss_{猫} = -0 \times log(0.1) - (1-0)log(1-0.1) = -log(0.9)$$
$$loss_{蛙} = -1 \times log(0.7) - (1-1)log(1-0.7) = -log(0.7)$$
$$loss_{鼠} = -1 \times log(0.8) - (1-1)log(1-0.8) = -log(0.8)$$

单张样本的loss即为$loss = loss_{猫} + loss_{蛙} + loss_{鼠}$

# 课后阅读

## 你真的完全理解logistic回归了吗

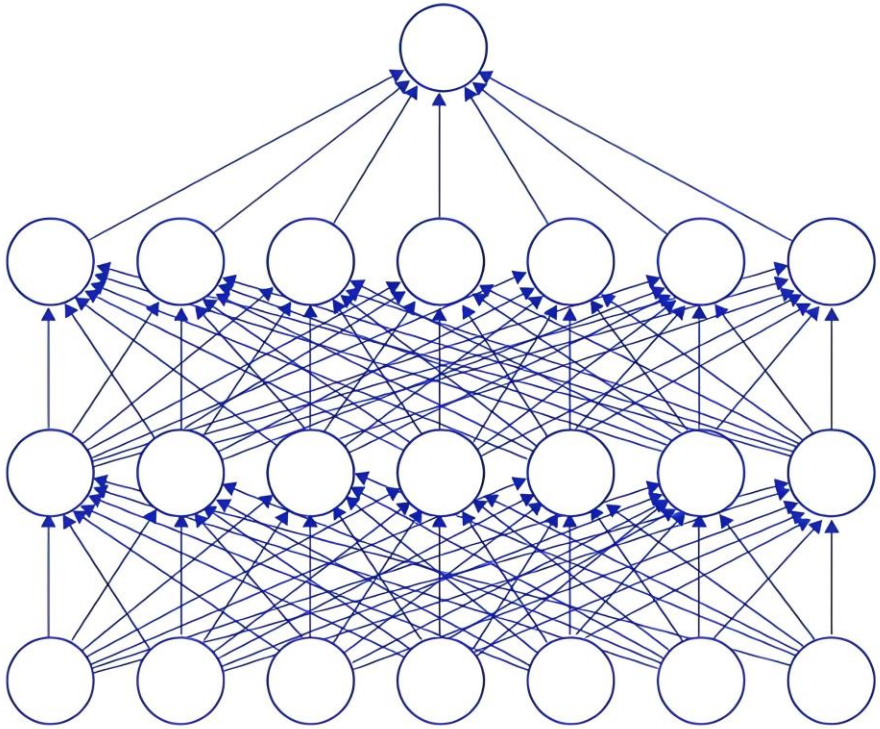
原创 小滔 机器学习实战 2024-04-17 11:42 广东

逻辑斯蒂回归(logistic regression)是最常见的机器学习算法之一，它可以用于预测事件发生的概率，例如基于给定的标记数据集预测一封来信是否为垃圾邮件或者一个肿瘤是否恶性。由于其简单性，逻辑斯蒂回归常被用作基准模型与其他更复杂的模型进行比较。该模型的名称中包含“逻辑”一词，因为它使用逻辑函数（sigmoid）将输入特征的线性组合转换为概率。逻辑斯蒂回归的输出是一个介于0和1之间的连续值，因此它的名称中还包含“回归”一词，但是通常作为二元分类器使用，通过选择一个阈值（通常为0.5），将概率大于阈值的输入分类为正类，低于阈值的分类为负类。

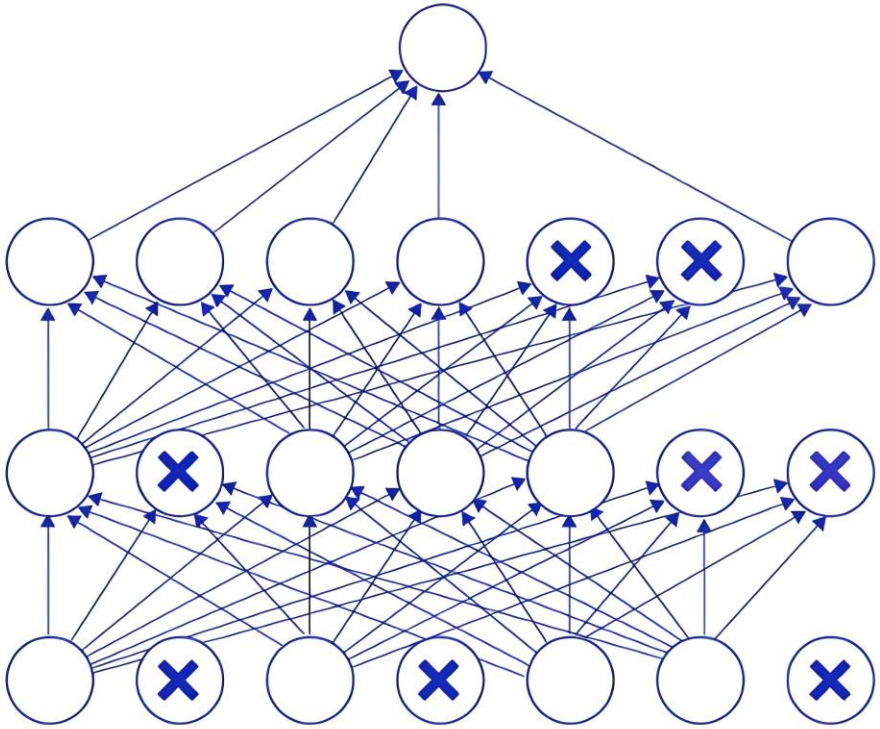
在这篇文章中，我们将深入讨论逻辑斯蒂回归模型，从零开始在Python中实现它，然后展示它在Scikit-Learn中的实现。在介绍该算法之前，我们需要考虑为什么不用线性回归进行分类。

**https://mp.weixin.qq.com/s/WS9k3vmO07i660cGYRX8ZA**

# Dropout

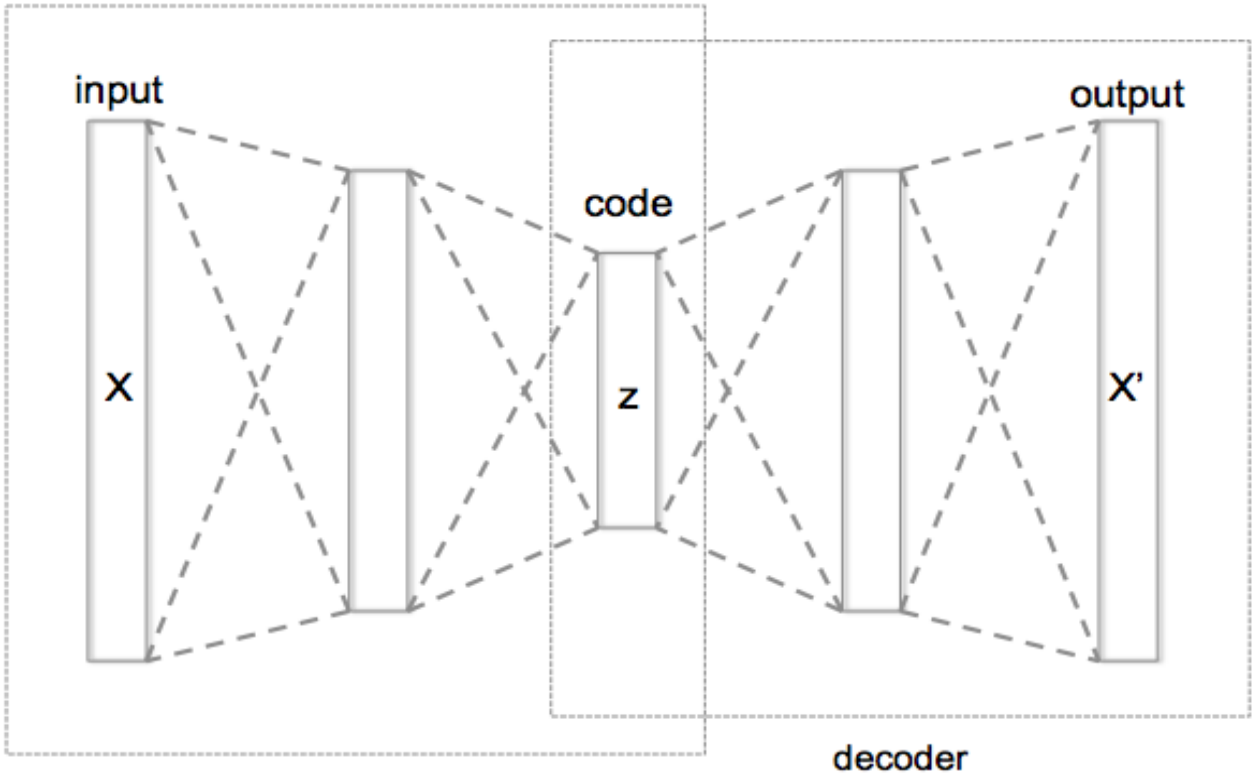


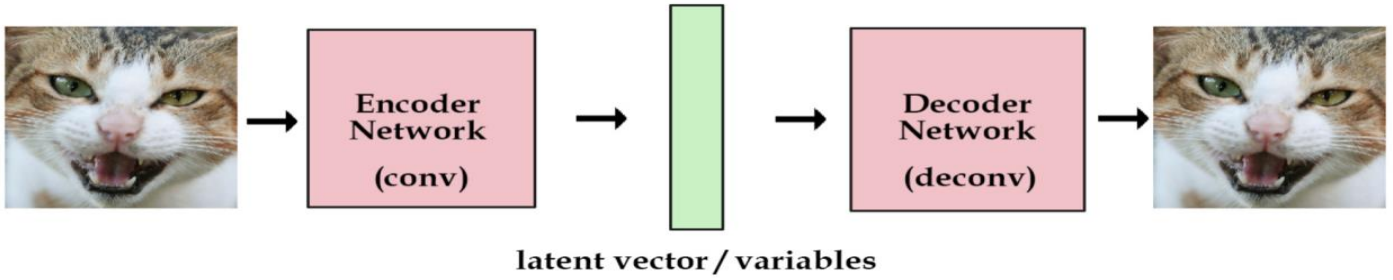
普通的神经网络



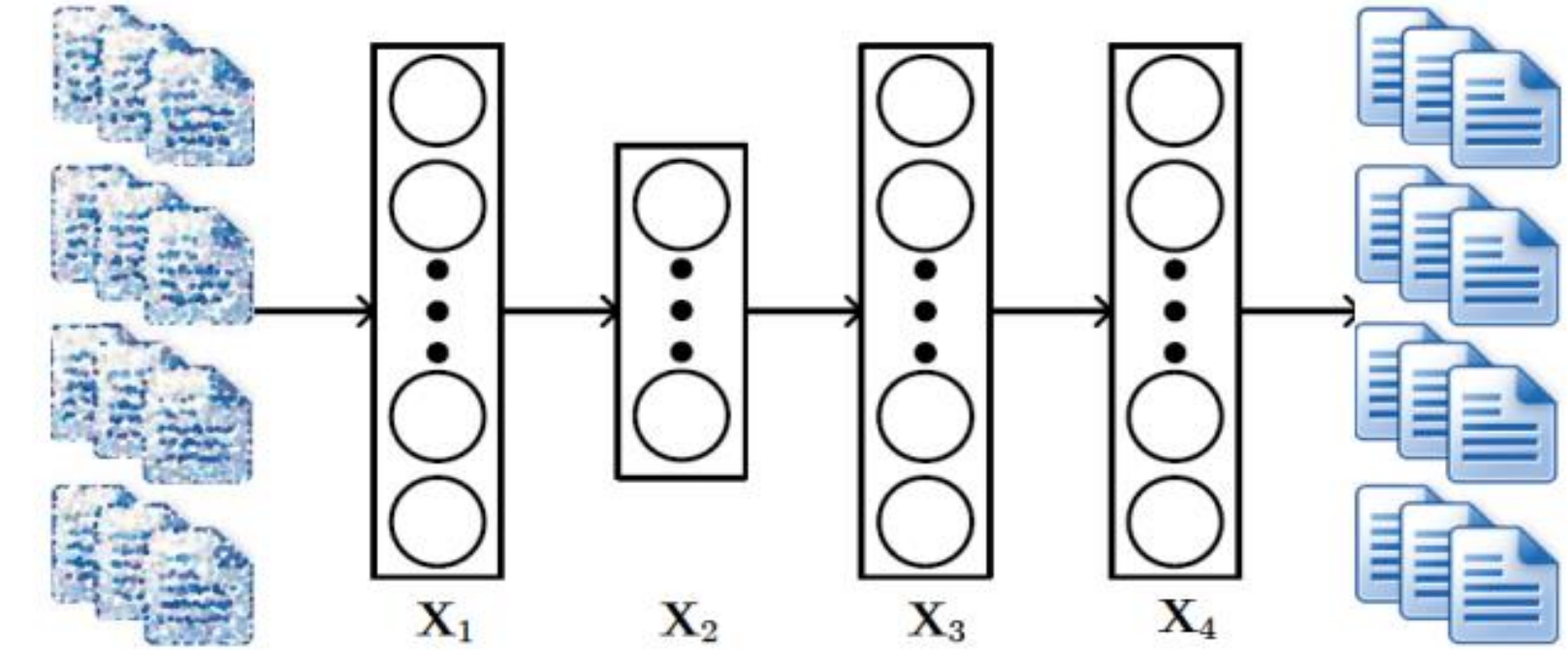
使用dropout的神经网络

# Auto-Encoders



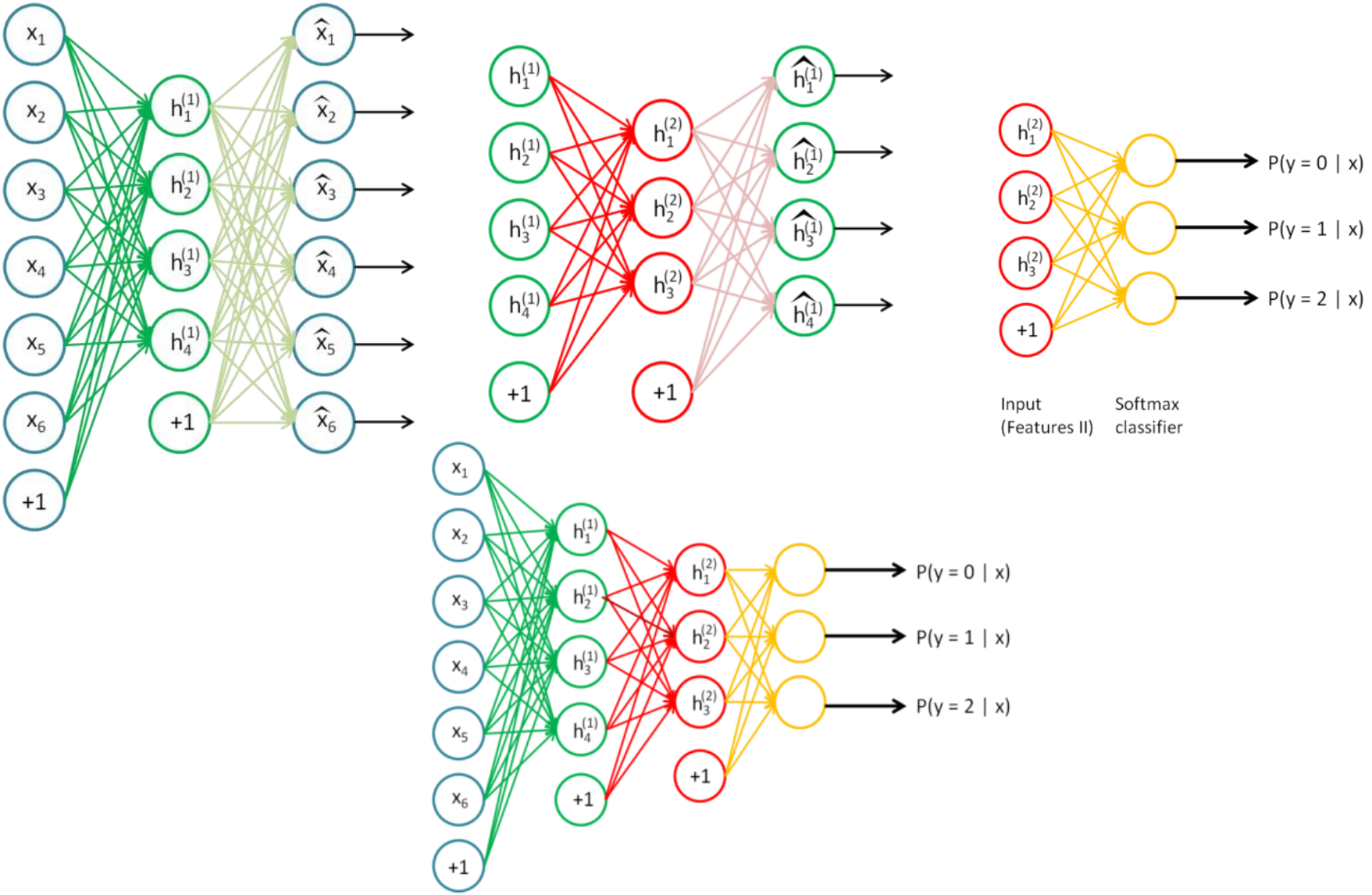
# Denoising Auto-Encoder



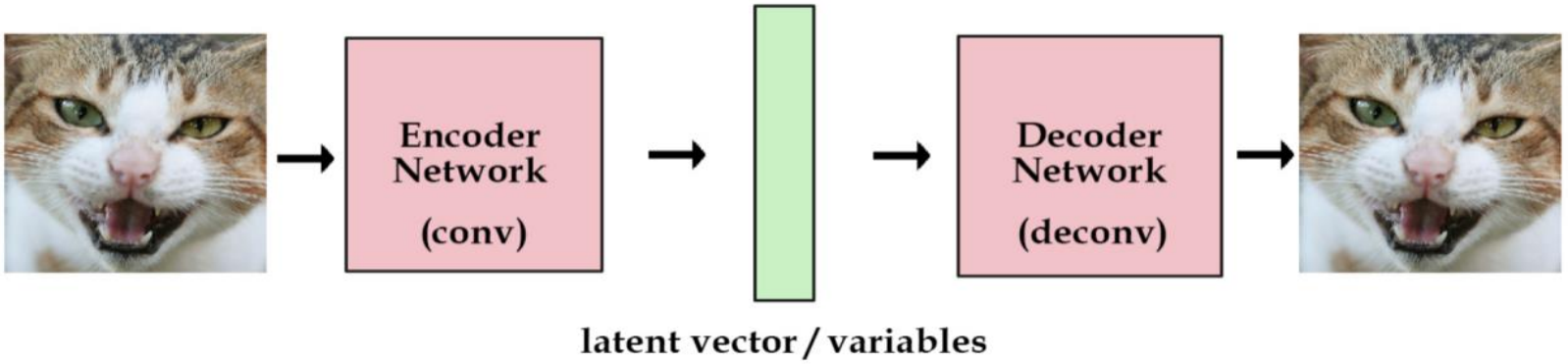

SDAE solves the following optimization problem:

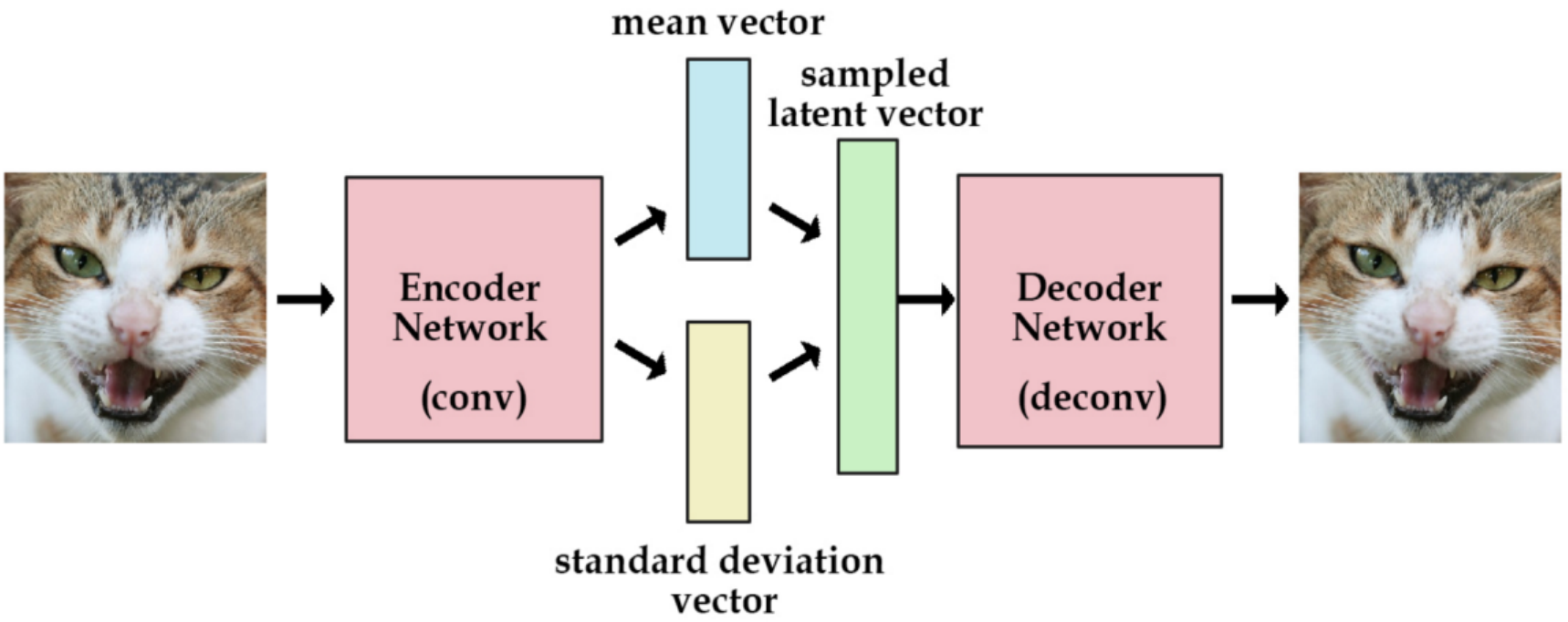
$$\min_{\{\mathbf{W}_l\},\{\mathbf{b}_l\}} \|\mathbf{X}_c - \mathbf{X}_L\|_F^2 + \lambda \sum_l \|\mathbf{W}_l\|_F^2,$$

# Stacked Auto-Encoders(SAE)

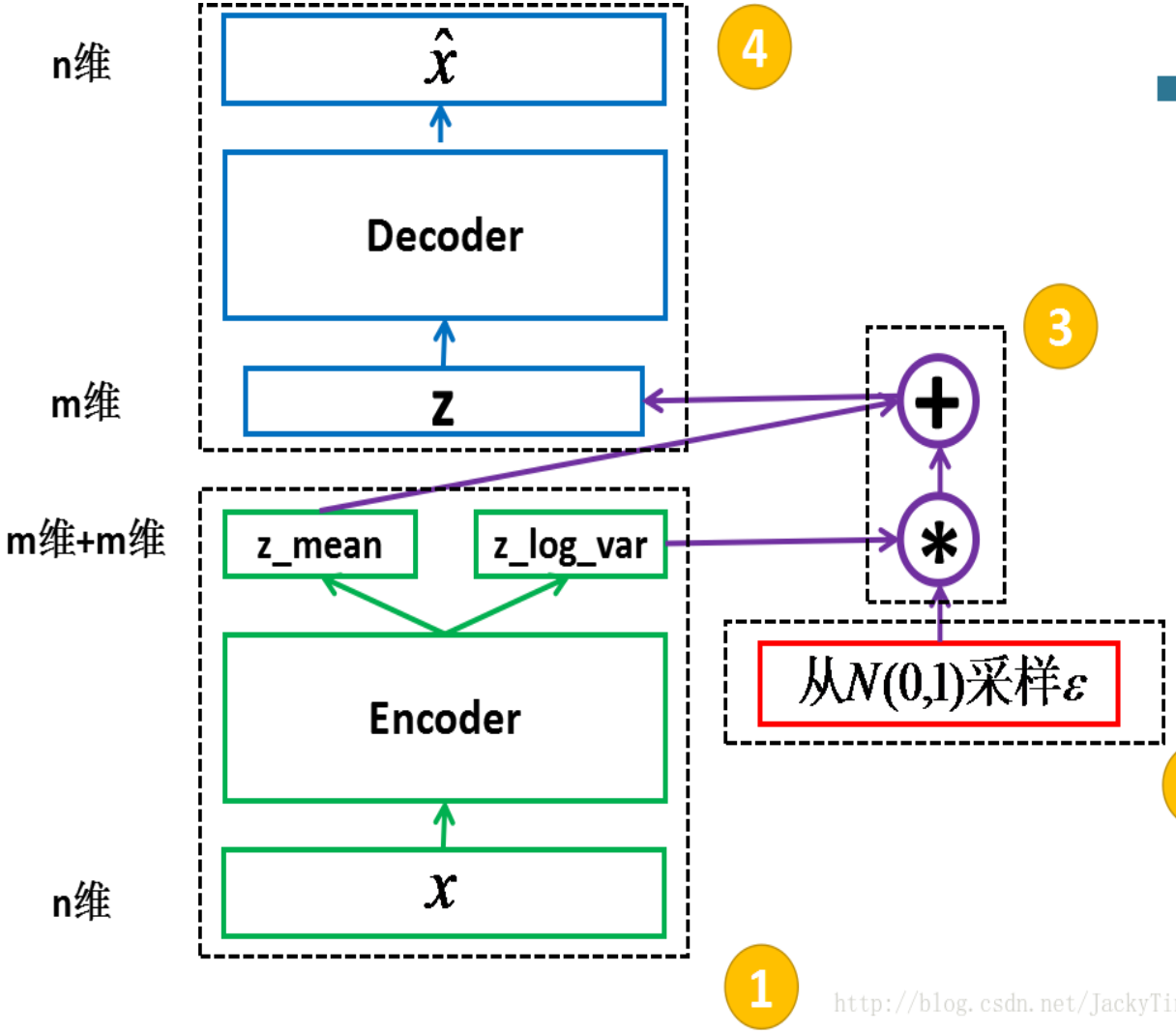


# Variational Autoencoder ( VAE )

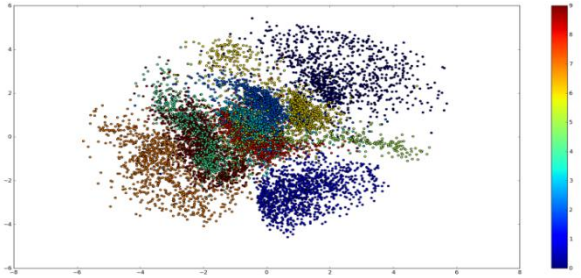


# Variational Autoencoder（VAE）




- VAE 有什么用？
  - 数据生成
  - 高维数据可视化
  - 缺失数据填补
  - 半监督学习

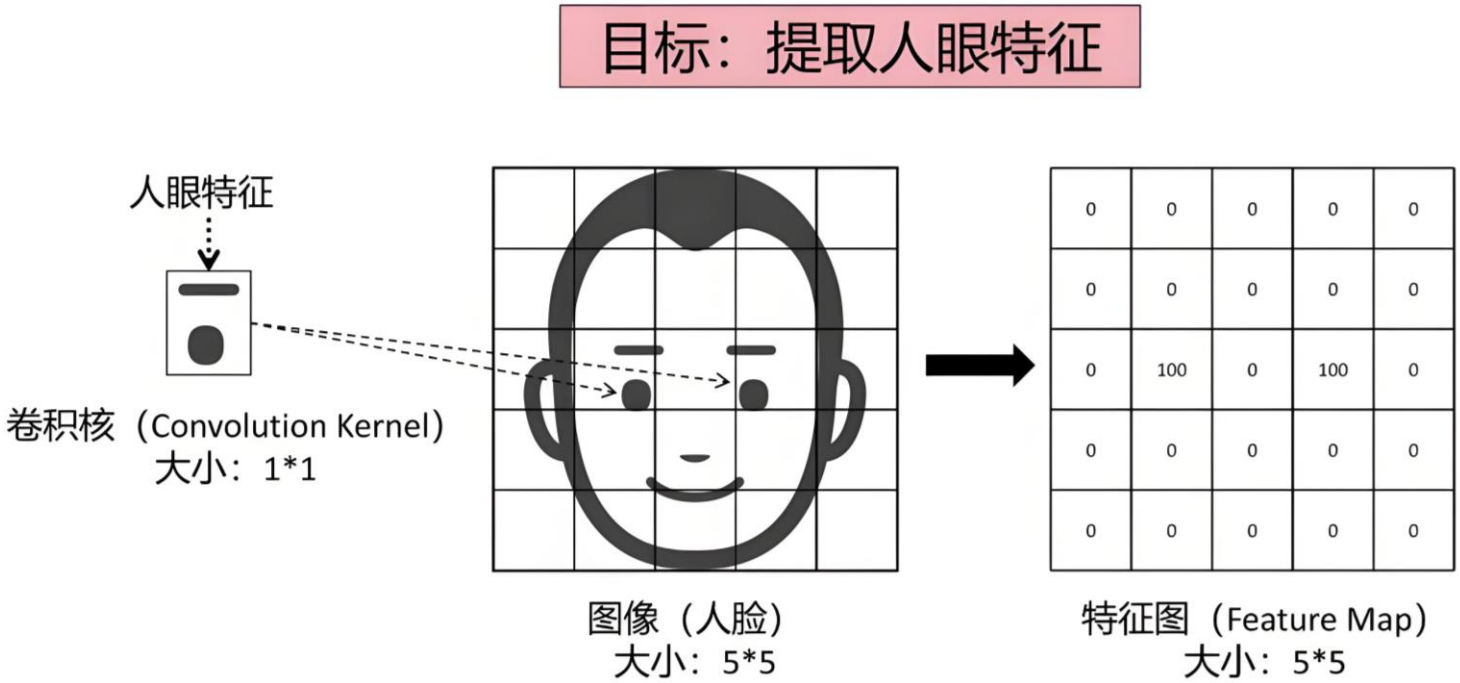


http://blog.csdn.net/JackyTintin



# 4. 经典深度学习算法

# Convolutional Neural Network



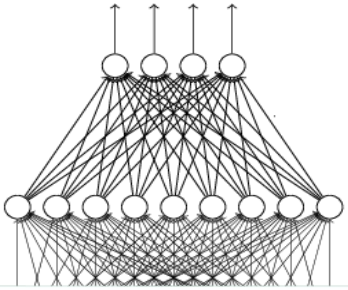



# 卷积神经网络

**Convolutional Neural Network**



**cat dog ……**



**Fully Connected Feedforward network**

**Convolution**

**Max Pooling**

**Convolution**

**Max Pooling**

**Can repeat many times**

**Flatten**

# CNN – Convolution

**Those are the network parameters to be learned.**

| | | | | | |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

**6 x 6 image**

| | | |
|---|---|---|
| 1 | -1 | -1 |
| -1 | 1 | -1 |
| -1 | -1 | 1 |

**Filter 1**

**Matrix**

| | | |
|---|---|---|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

**Filter 2**

**Matrix**

⋮ ⋮

**Property 1**

**Each filter detects a small pattern (3 x 3).**

| | | |
|---|---|---|
| 1 | -1 | -1 |
| -1 | 1 | -1 |
| -1 | -1 | 1 |

**Filter 1**

**If stride=2**

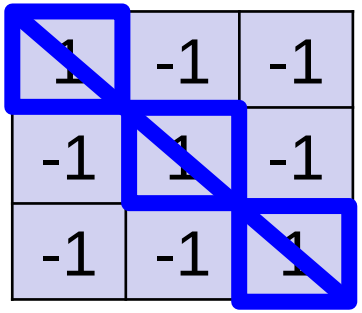
| | | | | | |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

**6 x 6 image**

**3** **-3**

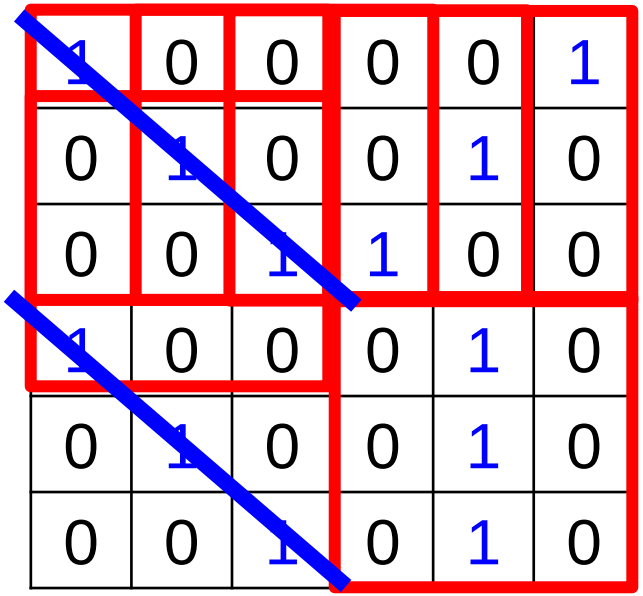
**We set stride=1 below**

# CNN – Convolution

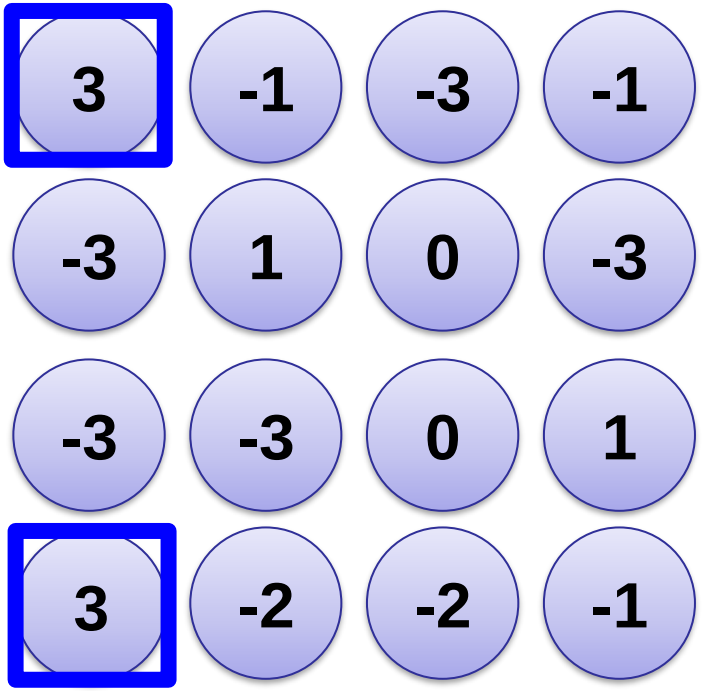



**Filter 1**

**stride=1**




**6 x 6 image**



# CNN – Convolution

| | | |
|---|---|---|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

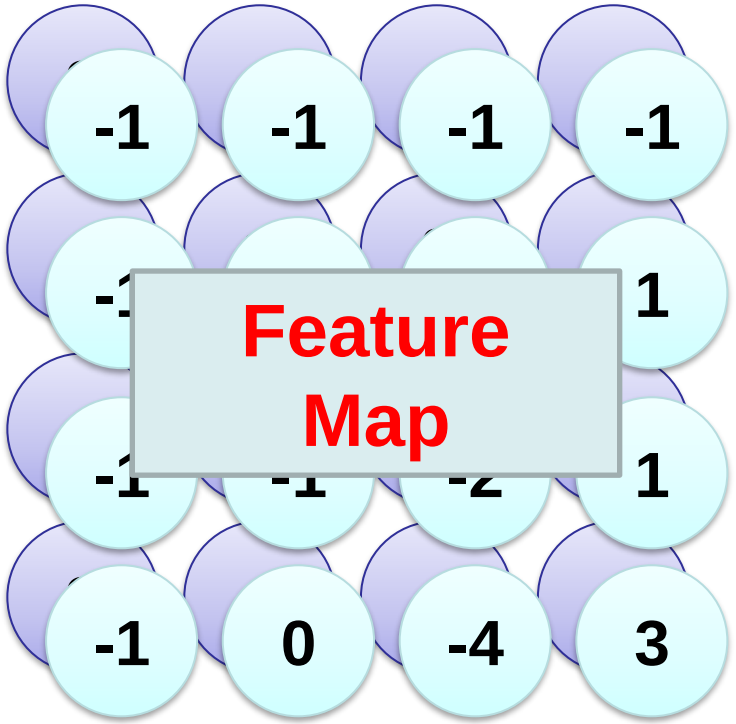
**Filter 2**

**stride=1**

| | | | | | |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

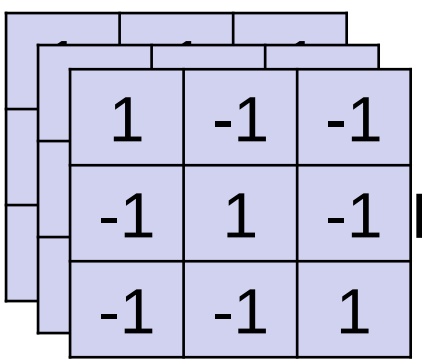
**6 x 6 image**

**Do the same process for every filter**

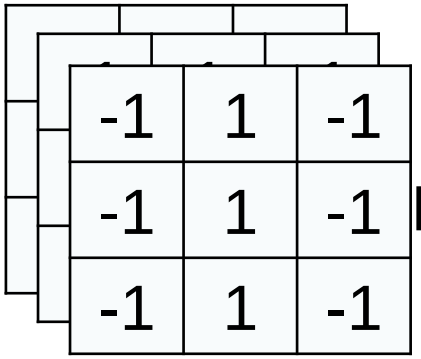



**4 x 4 image**

# CNN – Colorful image

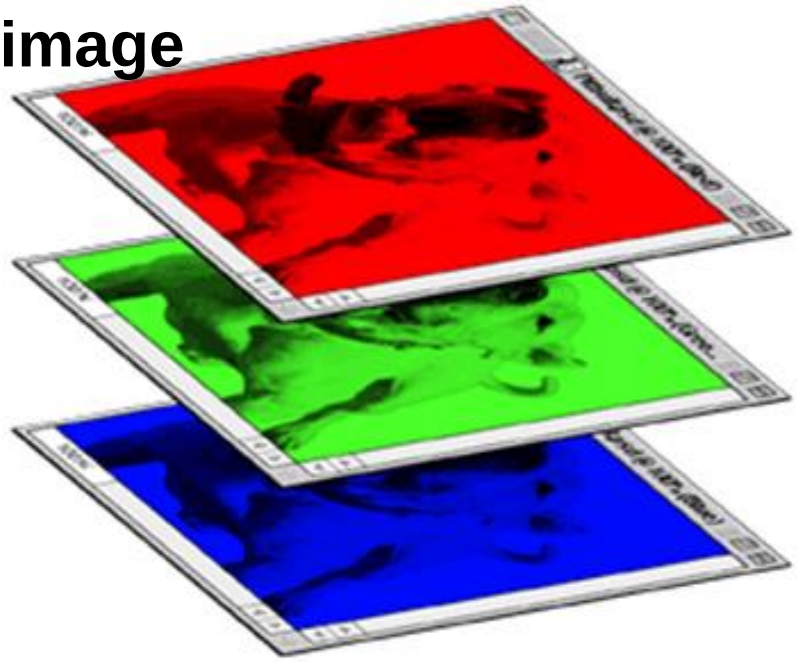


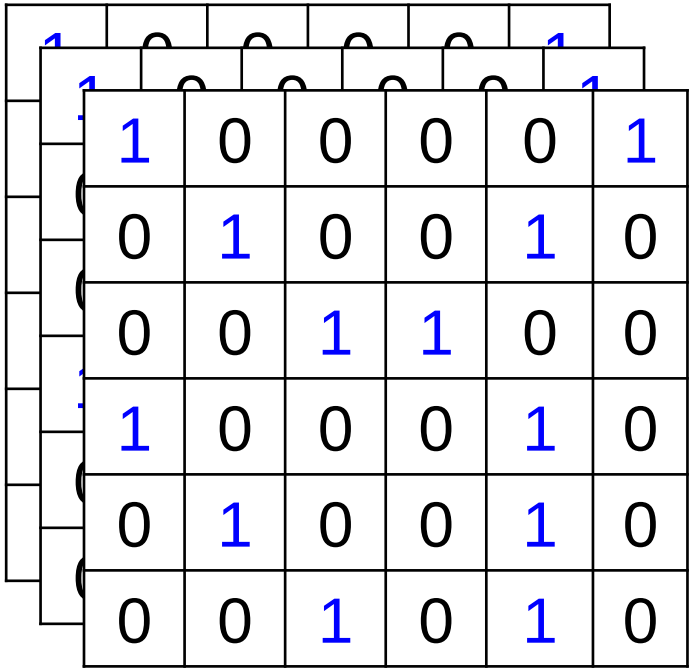

Filter 1



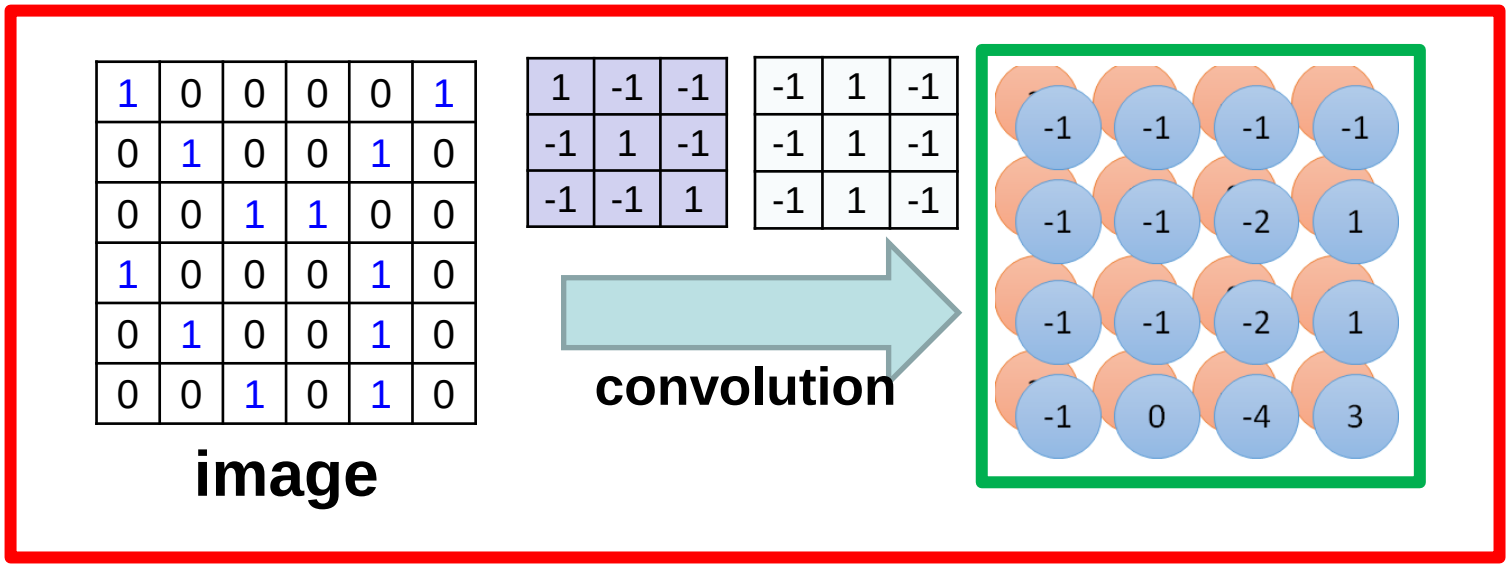

Filter 2

Colorful image

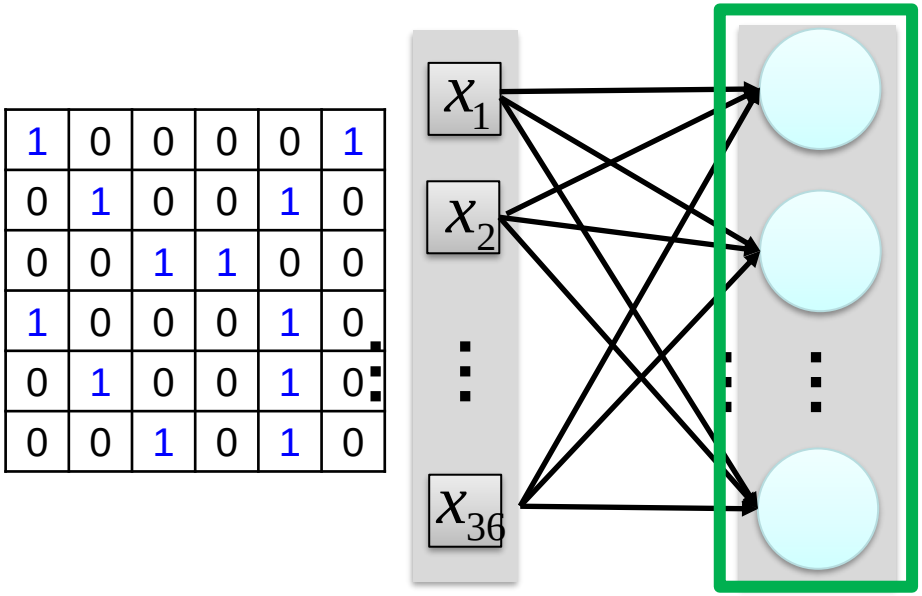


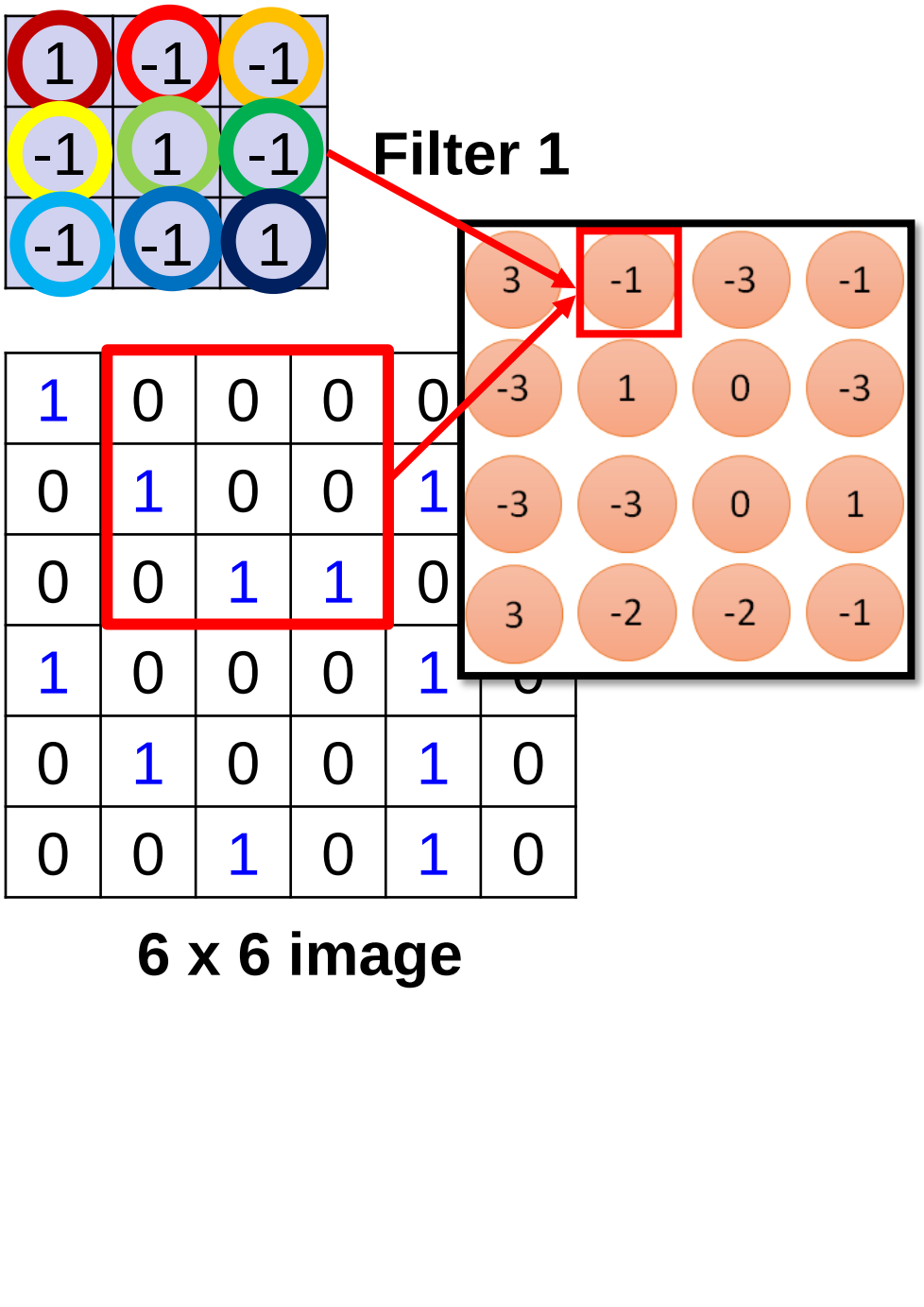


# Convolution v.s. Fully Connected




**Fully-connected**




1 -1 -1
-1 1 -1
-1 -1 1
Filter 1
3 -1 -3 -1
-3 1 0 -3
-3 -3 0 1
3 -2 -2 -1
1 0 0 0 0
0 1 0 0 1
0 0 1 1 0
1 0 0 0 1
0 1 0 0 1 0
0 0 1 0 1 0
6 x 6 image
1
2
3
4
7
8
9
10
13
14
15
16
3
-1
Shared weights

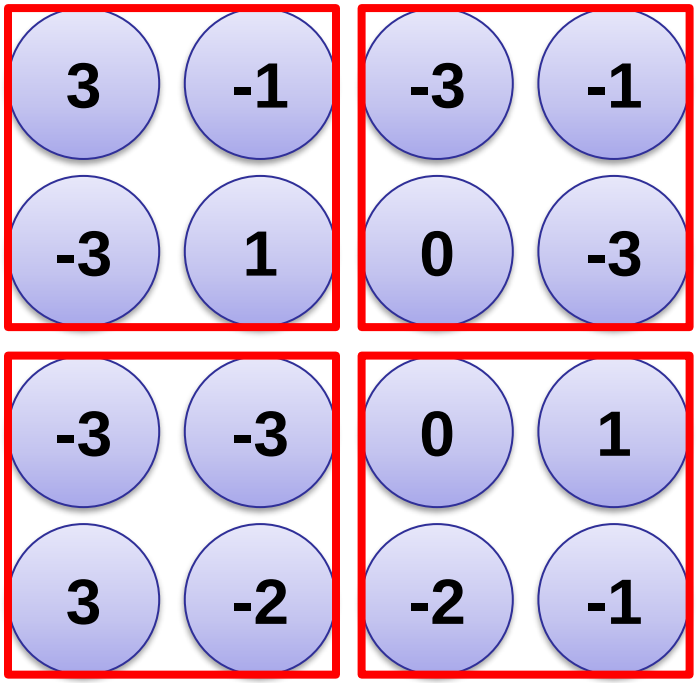
# CNN – Max Pooling

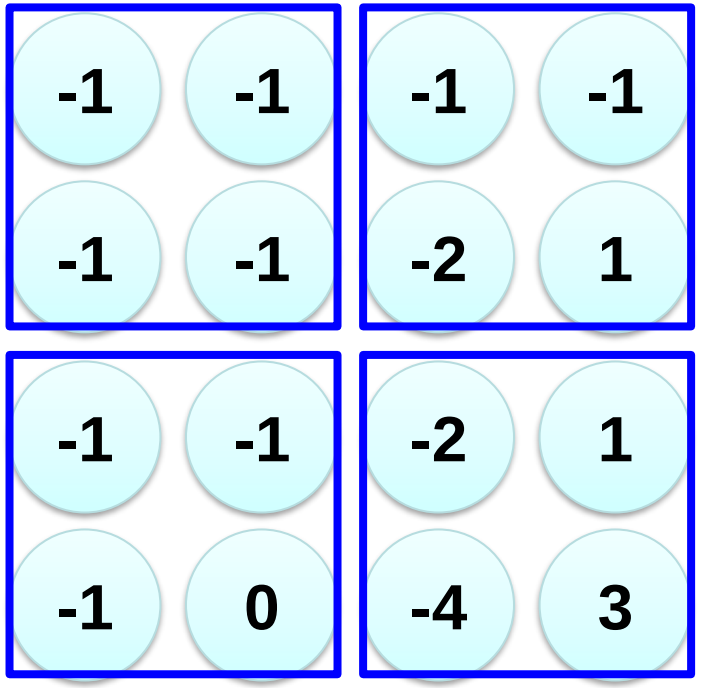
| | | |
|---|---|---|
| 1 | -1 | -1 |
| -1 | 1 | -1 |
| -1 | -1 | 1 |

**Filter 1**

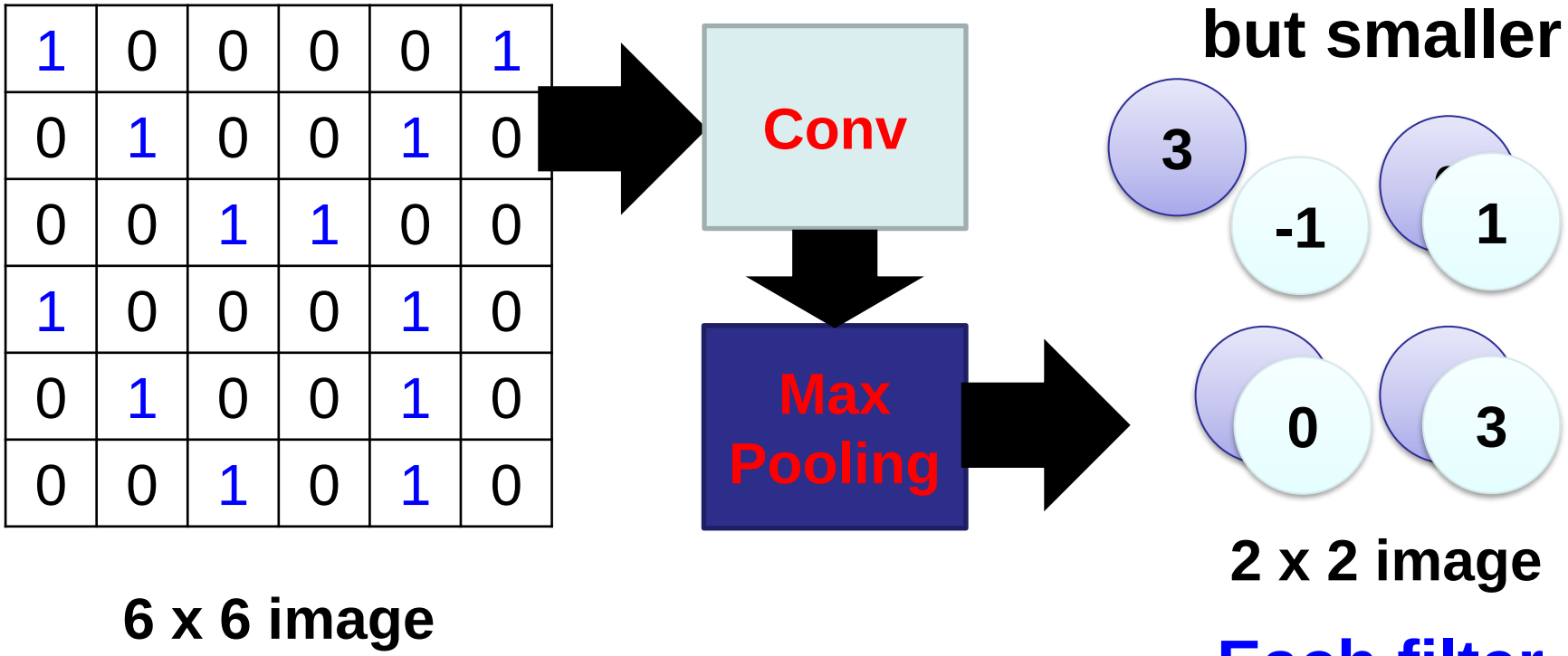
| | | |
|---|---|---|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

**Filter 2**

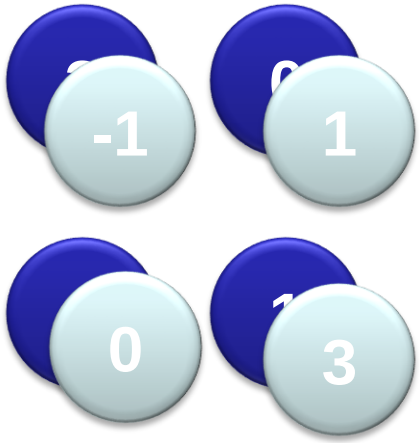


# CNN – Max Pooling

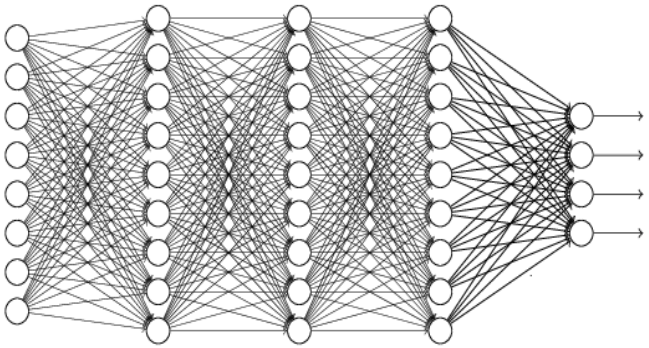


# Flatten



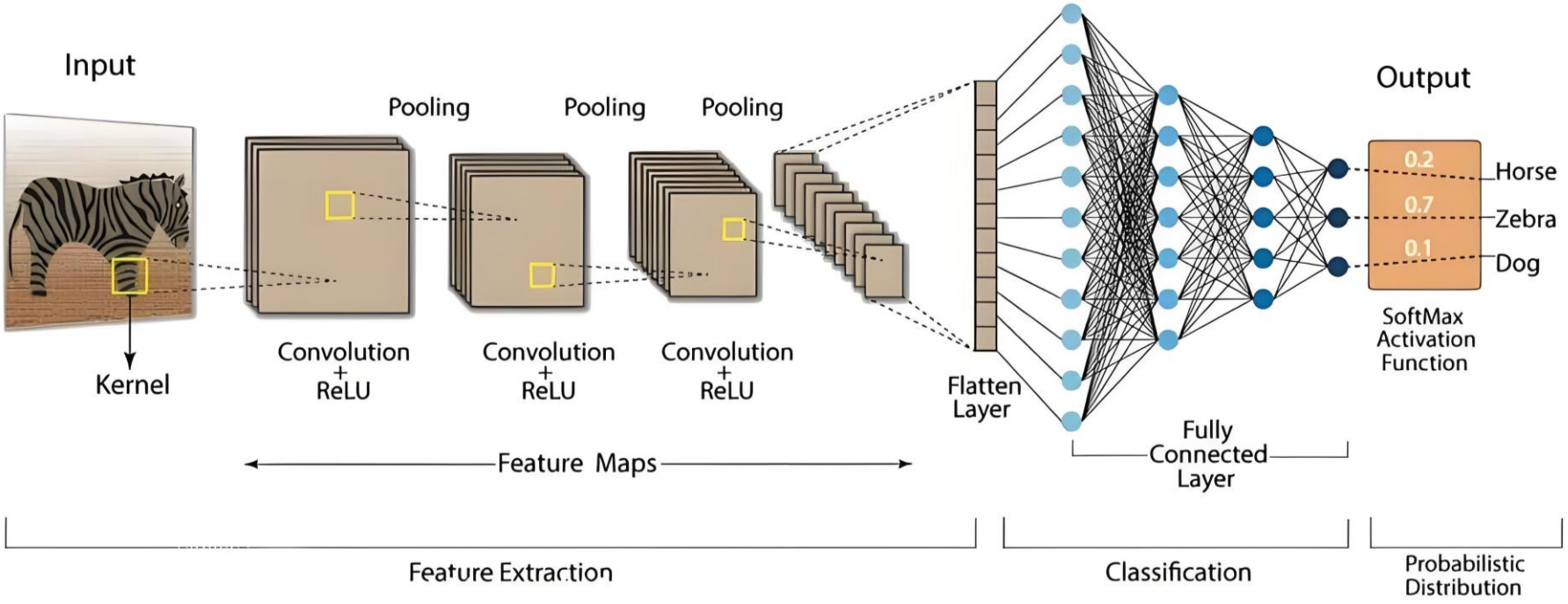

**Flatten**

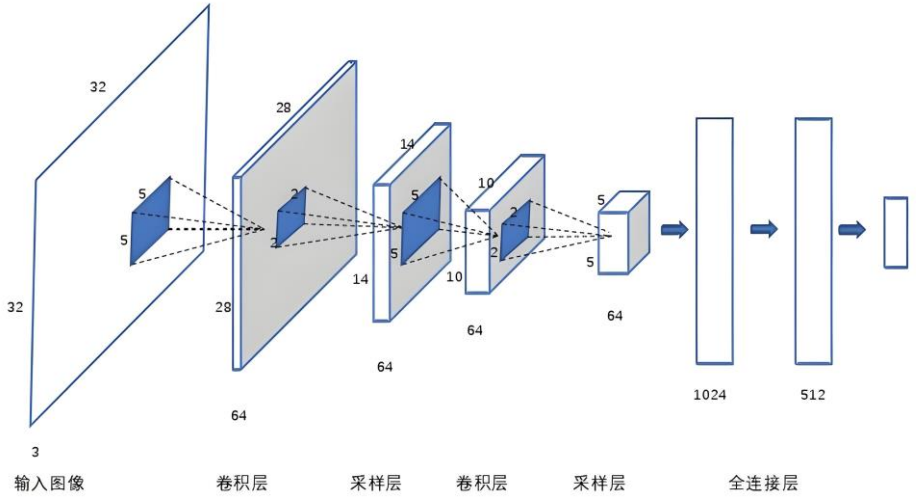


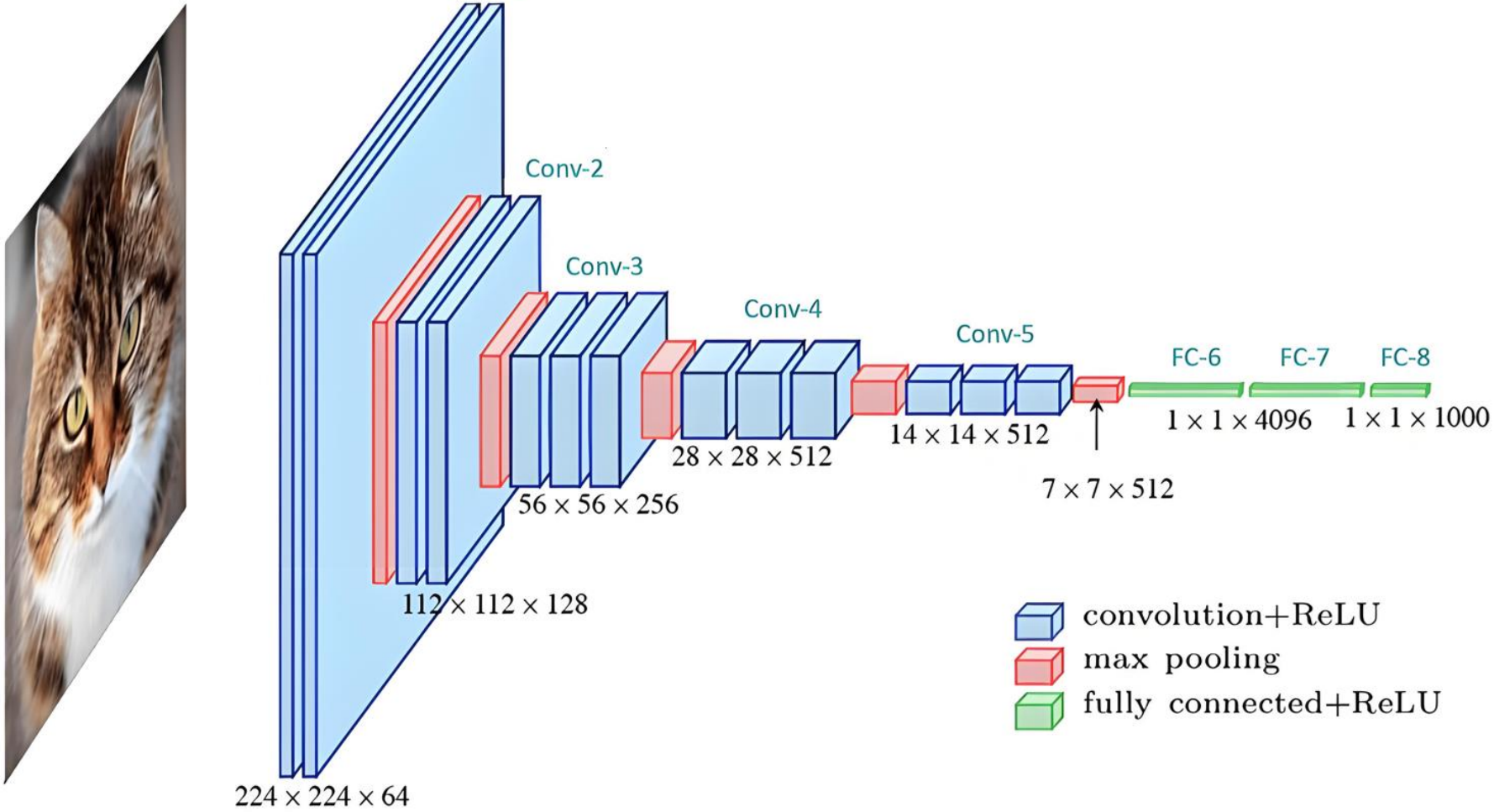
**Fully Connected Feedforward network**

# 卷积神经网络总结


Input
Pooling
Pooling
Pooling
Output
0.2
0.7
0.1
Horse
Zebra
Dog
Kernel
Convolution + ReLU
Convolution + ReLU
Convolution + ReLU
Flatten Layer
SoftMax Activation Function
Feature Maps
Fully Connected Layer
Feature Extraction
Classification
Probabilistic Distribution

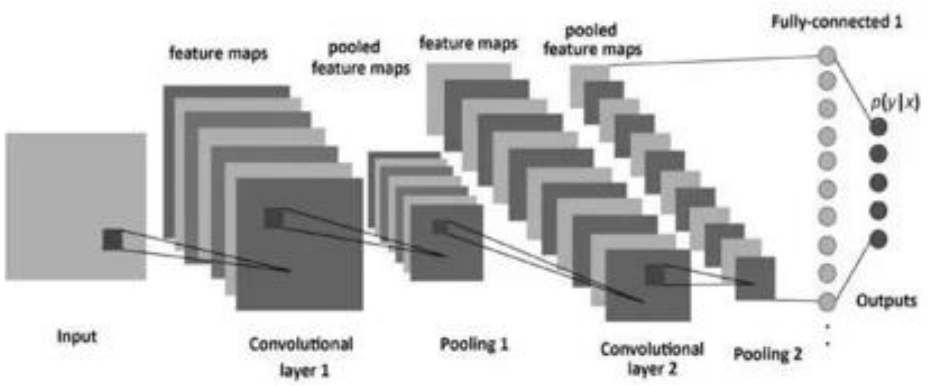

32
32
3
5
5
28
28
64
14
14
64
10
10
64
5
5
64
1024
512
输入图像
卷积层
采样层
卷积层
采样层
全连接层

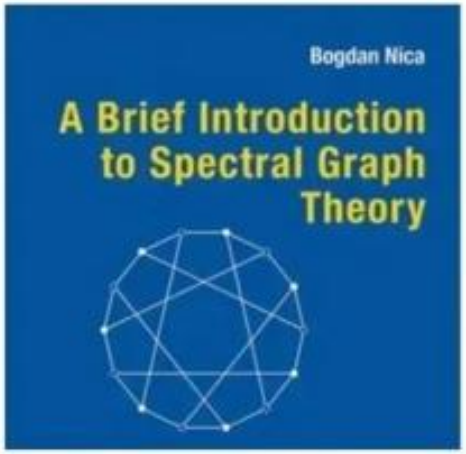
# 具体模型


Conv-1
Conv-2
Conv-3
Conv-4
Conv-5
FC-6
FC-7
FC-8
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU

# 图卷积网络

## Graph Convolution Networks (GCN)



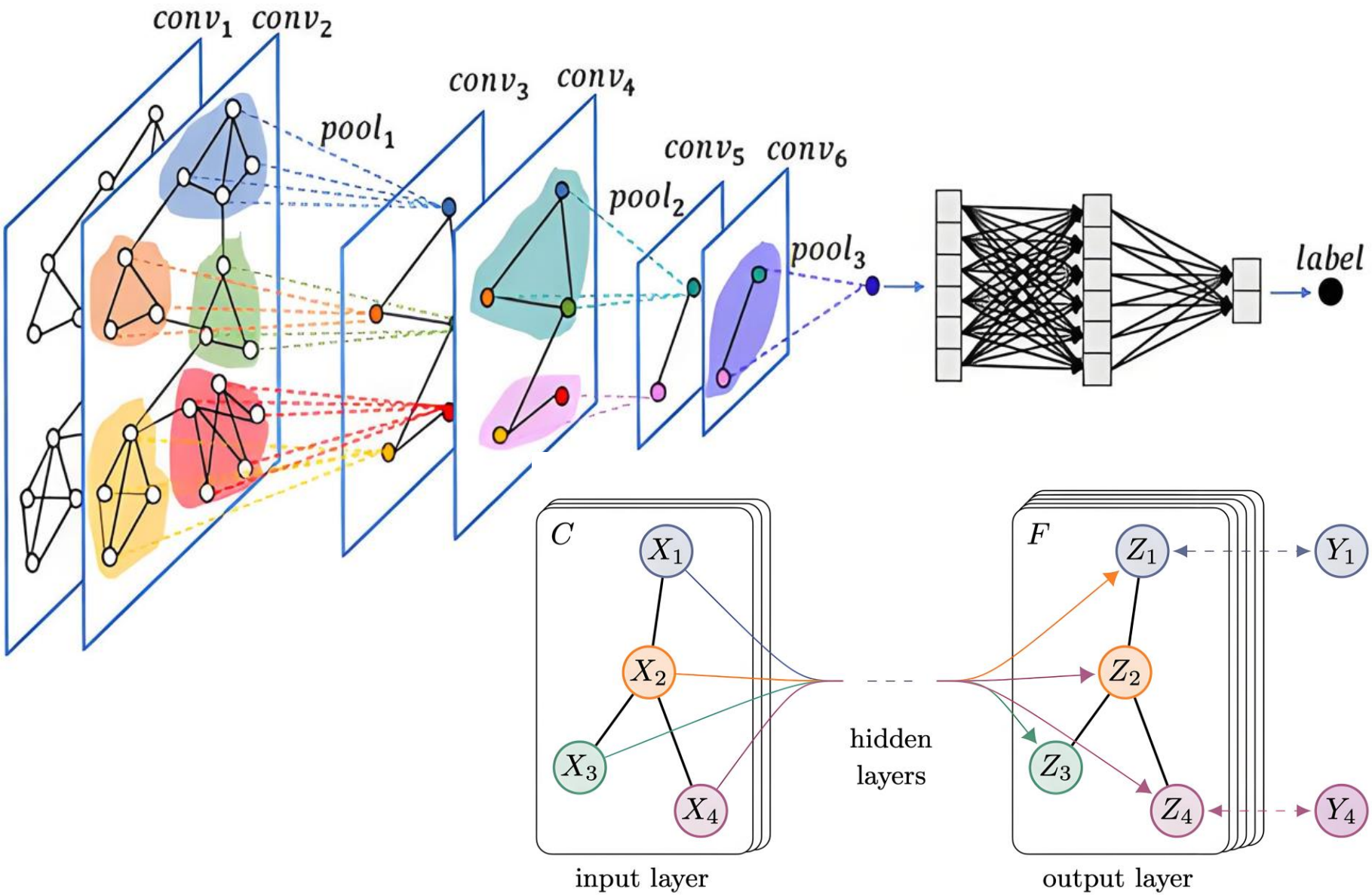

$$h_v = f\left(\frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} W x_u + b\right)$$

$$h_v^{k+1} = f\left(\frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} W^k h_u^k + b^k\right)$$

$$H^{(l+1)} = \sigma\left(D^{-\frac{1}{2}}\tilde{A}D^{-\frac{1}{2}}H^{(l)}W^{(l)}\right)$$

Semi-Supervised Classification with Graph Convolutional Networks (ICLR2017)

# 图卷积网络

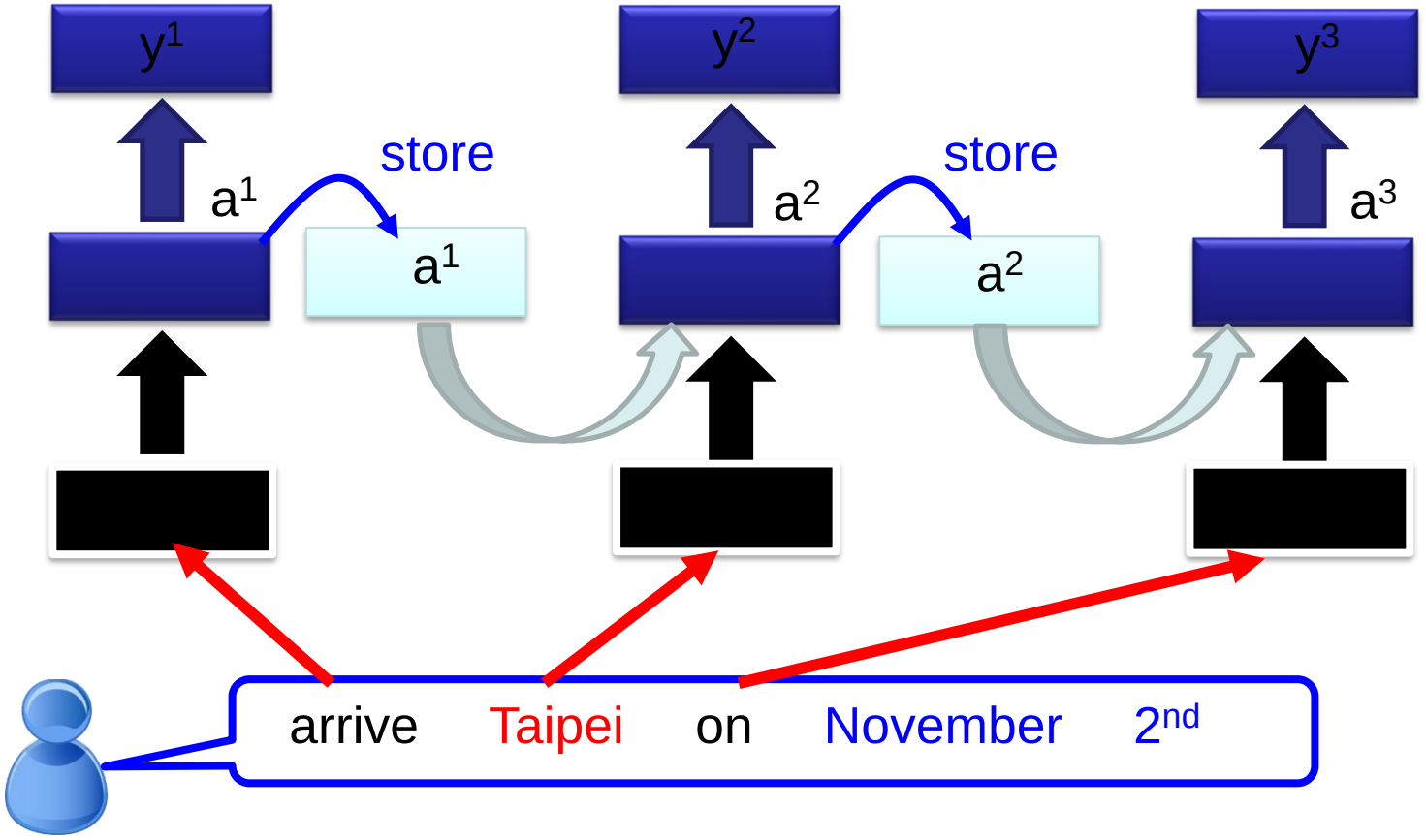

$conv_1$ $conv_2$
$pool_1$
$conv_3$ $conv_4$
$pool_2$
$conv_5$ $conv_6$
$pool_3$
label
C
$X_1$
$X_2$
$X_3$
$X_4$
input layer
hidden layers
F
$Z_1$
$Z_2$
$Z_3$
$Z_4$
$Y_1$
$Y_4$
output layer

# Recurrent Neural Network (RNN)

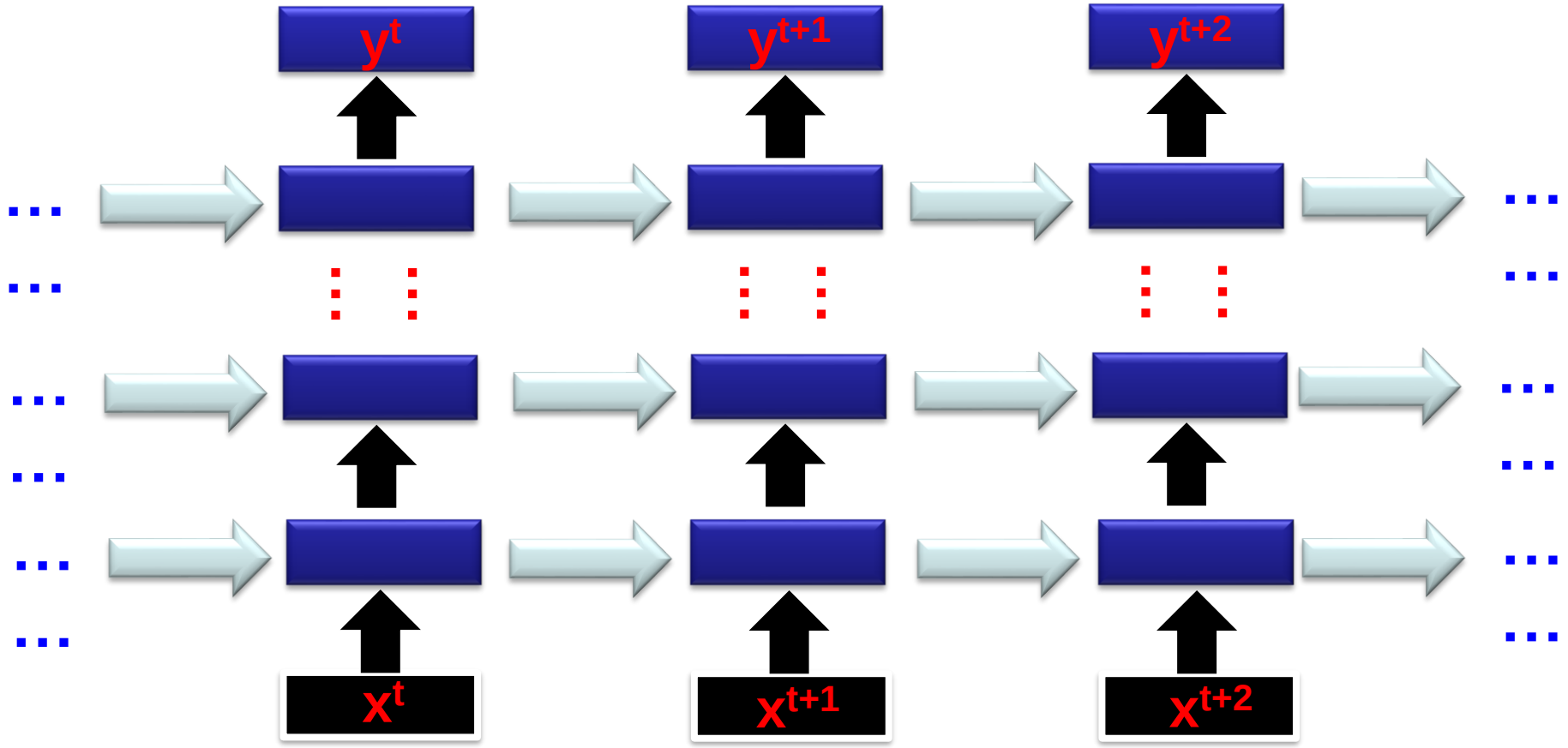
Probability of "arrive" in each slot

Probability of "Taipei" in each slot

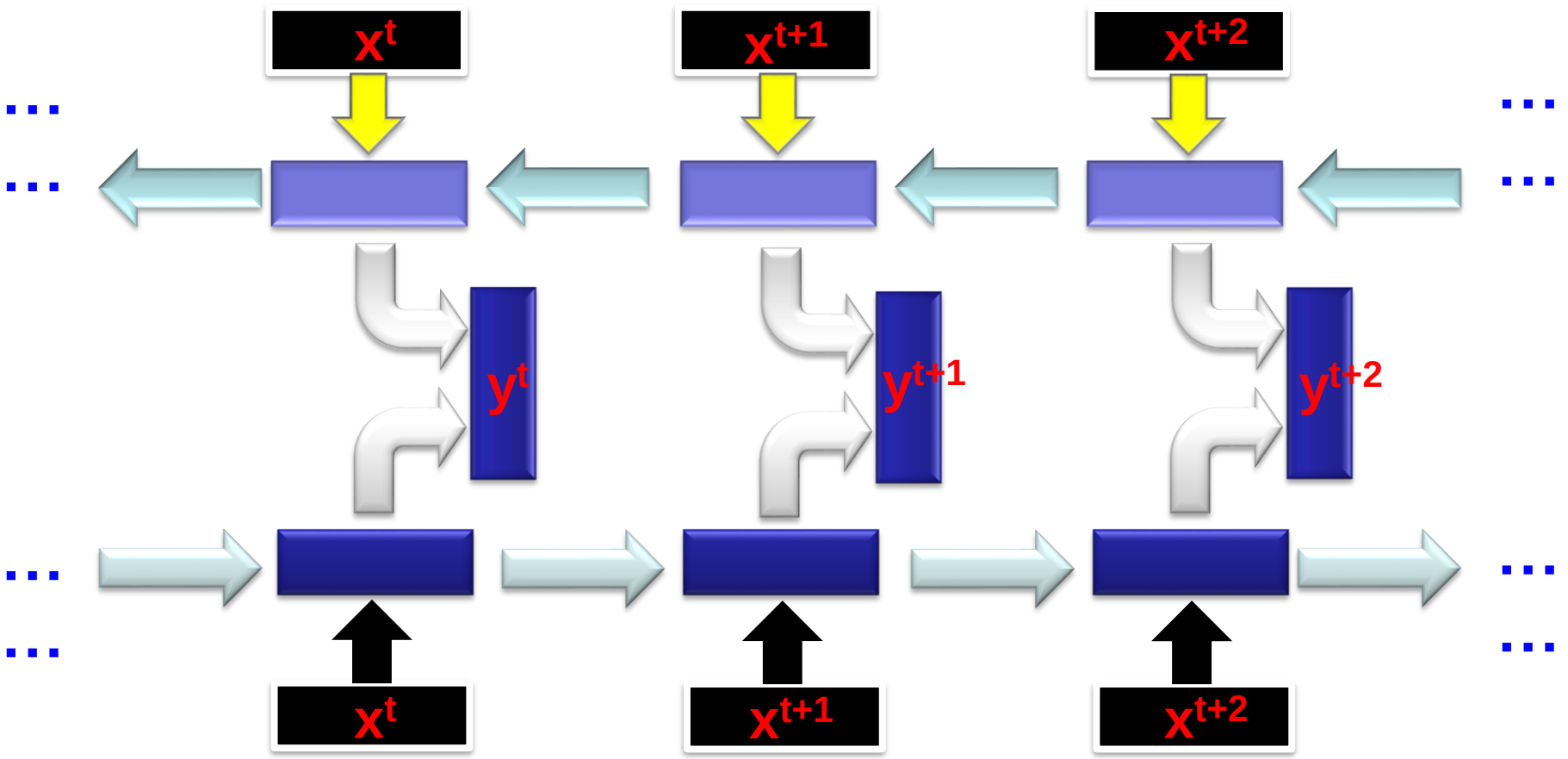
Probability of "on" in each slot



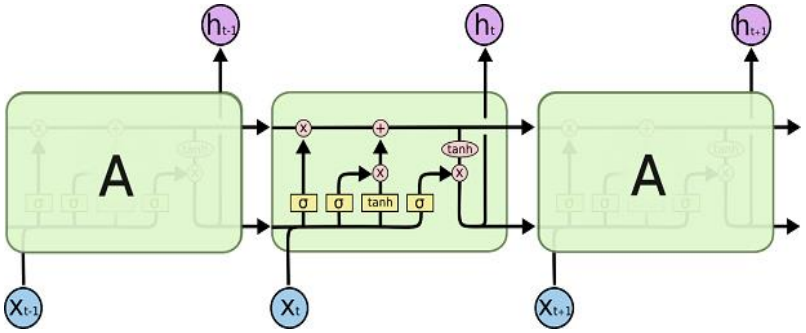
# Of course it can be deep …



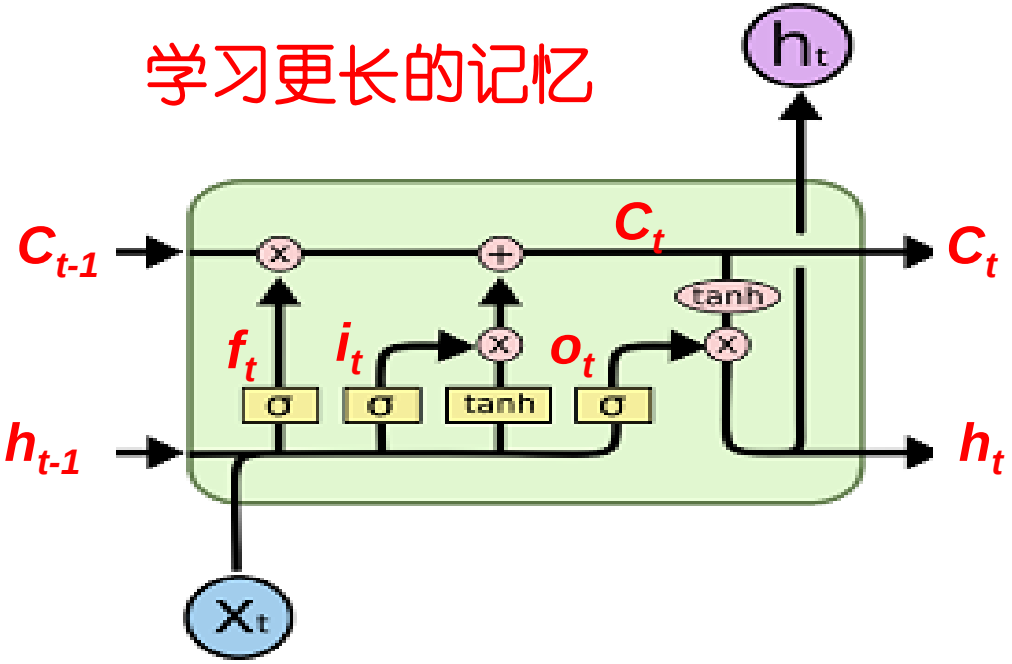
# Bidirectional RNN

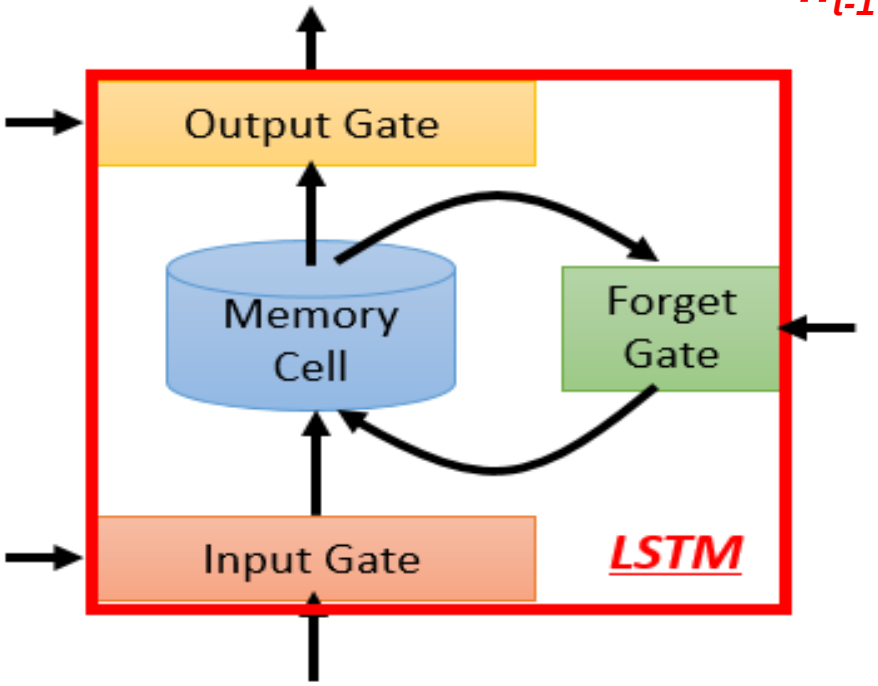


# 长短记忆网络 (LSTM)




学习更长的记忆




$$i_t = \delta(W_i \cdot [h_{t-1}, x_t] + b_i)$$

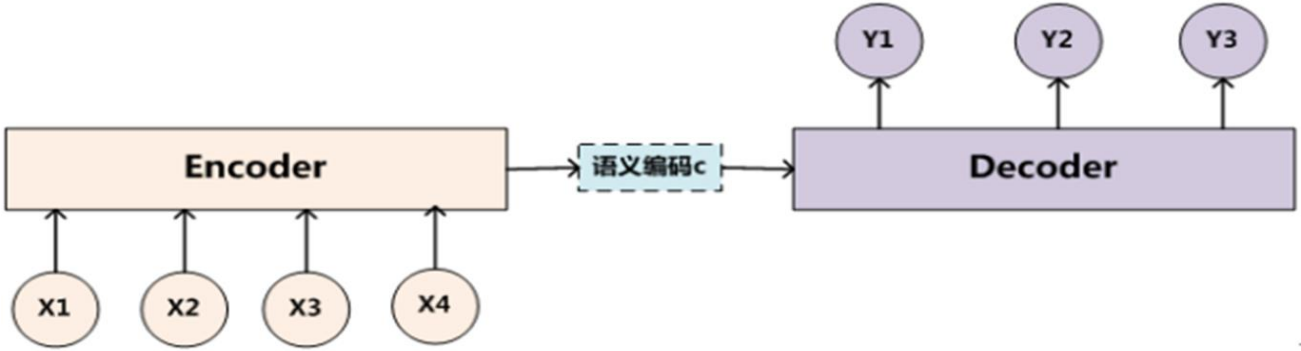
$$f_t = \delta(W_f \cdot [h_{t-1}, x_t] + b_f)$$

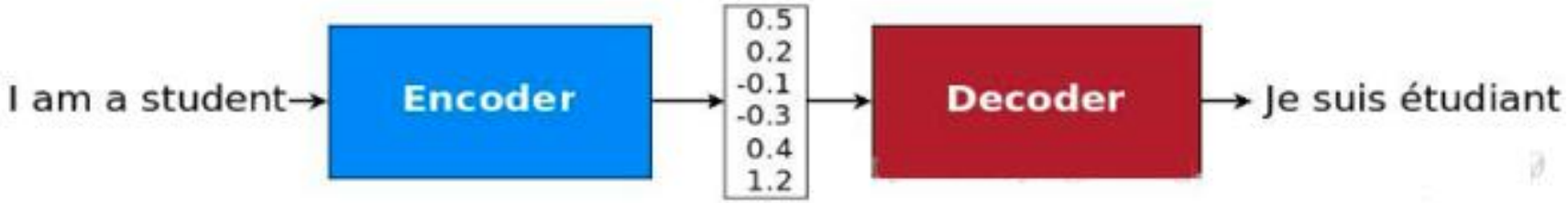
$$o_t = \delta(W_o \cdot [h_{t-1}, x_t] + b_o)$$

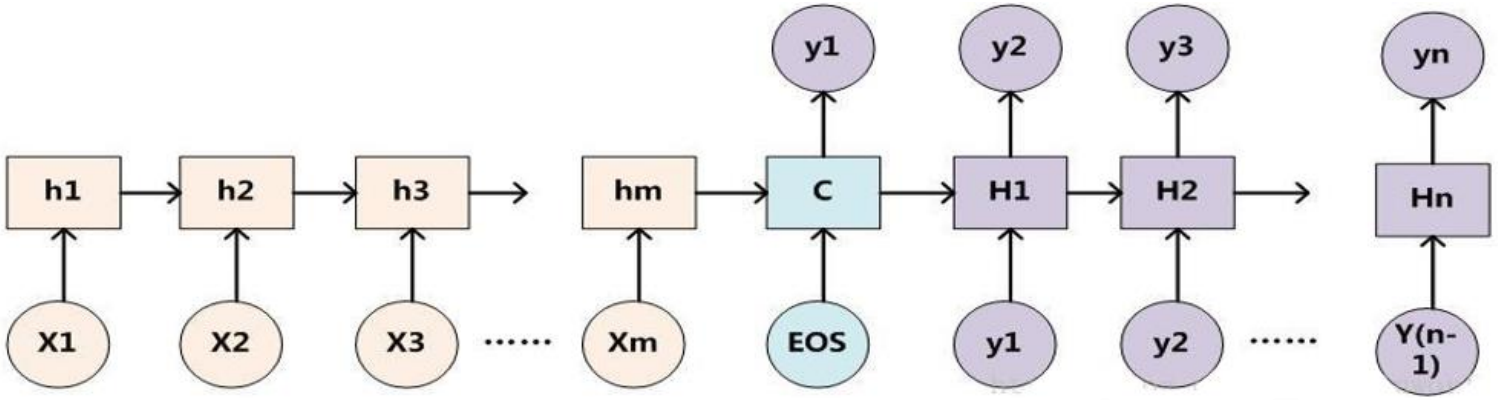
$$C_t = i_t * tanh(W_C \cdot [h_{t-1}, x_t] + b_C) + f_t * C_{t-1}$$

$$h_t = o_t * tanh(C_t)$$

# Seq2Seq



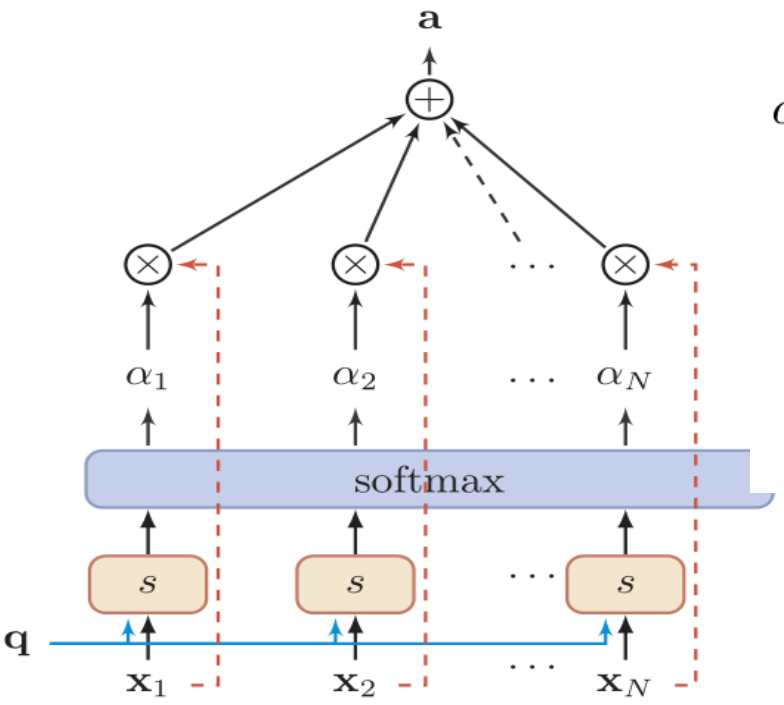



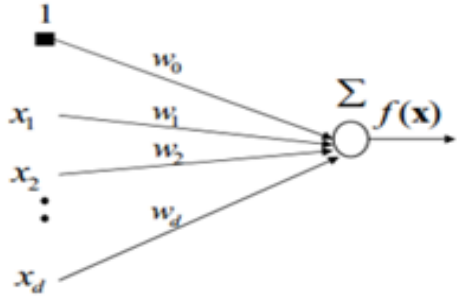




$$\mathbf{h}_{\langle t \rangle} = f\left(\mathbf{h}_{\langle t-1 \rangle}, y_{t-1}, \mathbf{c}\right),$$

$$\max_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^{N} \log p_{\boldsymbol{\theta}}(\mathbf{y}_n \mid \mathbf{x}_n),$$

# Attention机制




**第一步**

$$\alpha_i = p(z=i|X,\mathbf{q}) = \text{softmax}\left(s(\mathbf{x}_i,\mathbf{q})\right) = \frac{\exp\left(s(\mathbf{x}_i,\mathbf{q})\right)}{\sum_{j=1}^{N}\exp\left(s(\mathbf{x}_j,\mathbf{q})\right)}$$

**第一步**

$$\mathbf{att}(X,\mathbf{q}) = \sum_{i=1}^{N}\alpha_i\mathbf{x}_i, = \mathbb{E}_{z\sim p(z|X,\mathbf{q})}[\mathbf{x}]$$

加性模型 $s(\mathbf{x}_i,\mathbf{q}) = \mathbf{v}^{\mathrm{T}}\tanh(W\mathbf{x}_i + U\mathbf{q})$,

点积模型 $s(\mathbf{x}_i,\mathbf{q}) = \mathbf{x}_i^{\mathrm{T}}\mathbf{q}$,

缩放点积模型 $s(\mathbf{x}_i,\mathbf{q}) = \frac{\mathbf{x}_i^{\mathrm{T}}\mathbf{q}}{\sqrt{d}}$,

双线性模型 $s(\mathbf{x}_i,\mathbf{q}) = \mathbf{x}_i^{\mathrm{T}}W\mathbf{q}$,

权重不是像普通神经网络直接学习得到

# Hierarchical Attention

## Sentence Attention

$$u_i = \tanh(W_s h_i + b_s),$$

$$\alpha_i = \frac{\exp(u_i^\top u_s)}{\sum_i \exp(u_i^\top u_s)},$$

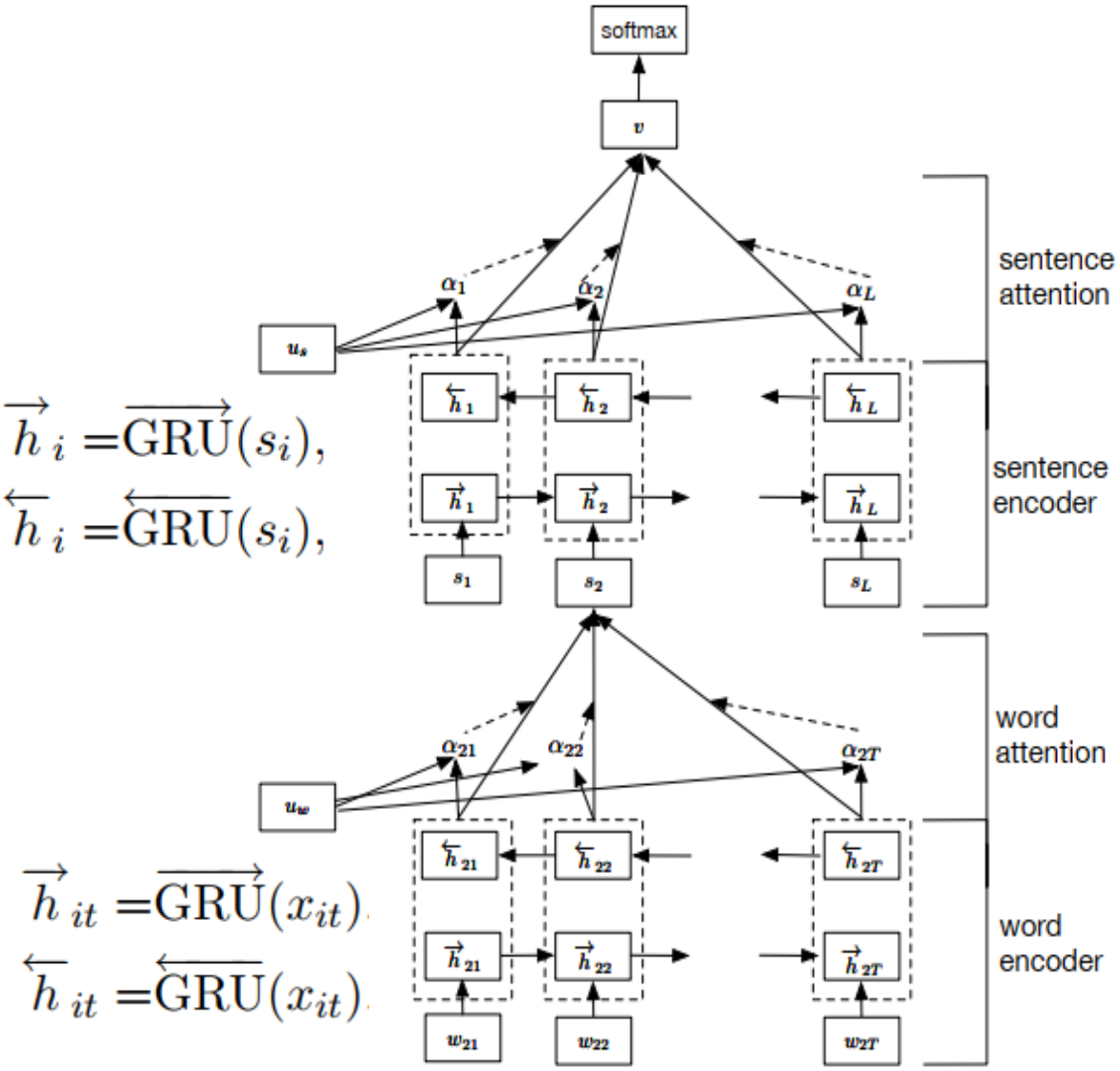
$$v = \sum_i \alpha_i h_i,$$

## Word Attention

$$u_{it} = \tanh(W_w h_{it} + b_w)$$

$$\alpha_{it} = \frac{\exp(u_{it}^\top u_w)}{\sum_t \exp(u_{it}^\top u_w)}$$

$$s_i = \sum_t \alpha_{it} h_{it}.$$




$$\overrightarrow{h}_i = \overrightarrow{\mathrm{GRU}}(s_i),$$

$$\overleftarrow{h}_i = \overleftarrow{\mathrm{GRU}}(s_i),$$

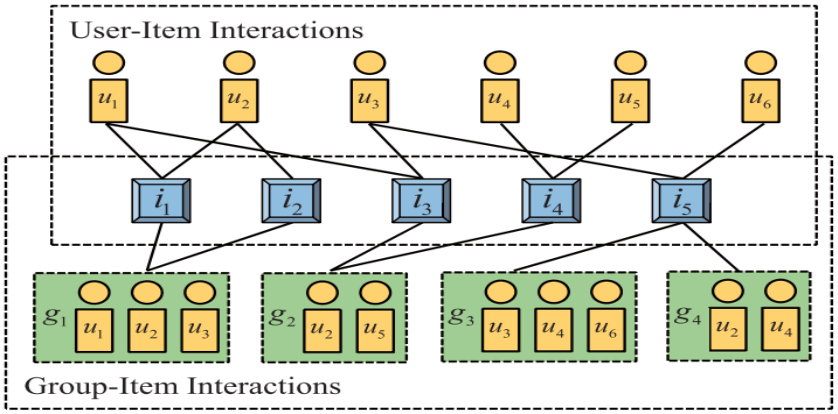
$$\overrightarrow{h}_{it} = \overrightarrow{\mathrm{GRU}}(x_{it})$$

$$\overleftarrow{h}_{it} = \overleftarrow{\mathrm{GRU}}(x_{it})$$

$$x_{it} = W_e w_{it}$$

Hierarchical Attention Networks for Document Classification (NAACL2016)

# Attentive Group

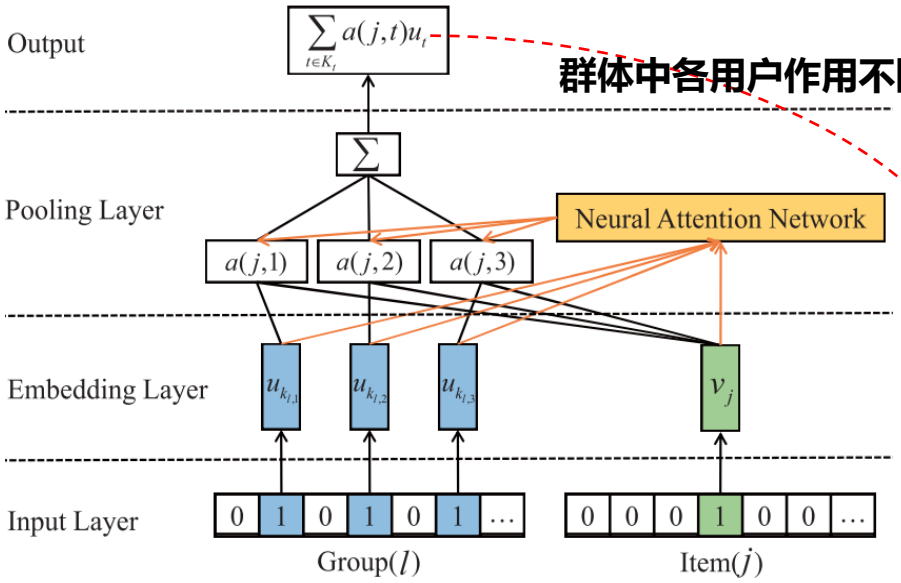



用户所取作用取决于与item的关系

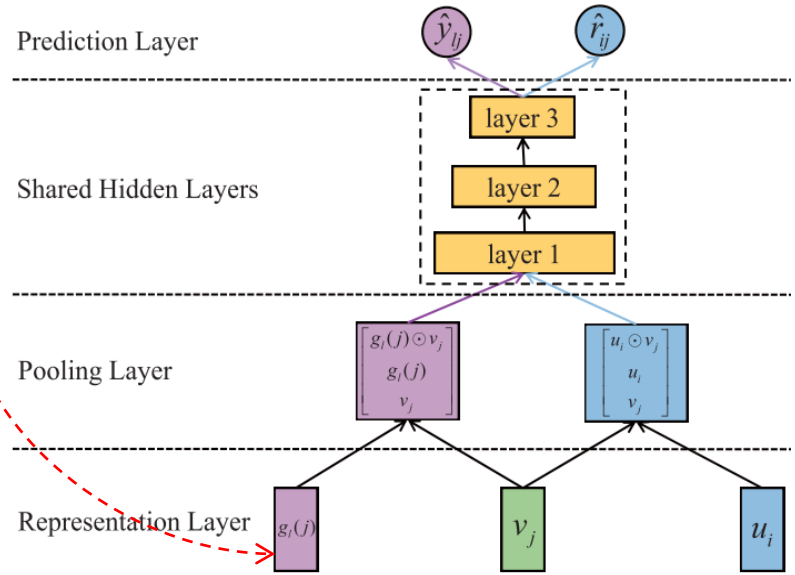
$$\mathcal{L}_{user} = \sum_{(i,j,s)\in O} (r_{ijs} - \hat{r}_{ijs})^2 = \sum_{(i,j,s)\in O} (\hat{r}_{ij} - \hat{r}_{is} - 1)^2$$

$$\mathcal{L}_{group} = \sum_{(l,j,s)\in O'} (y_{ljs} - \hat{y}_{ljs})^2 = \sum_{(l,j,s)\in O'} (\hat{y}_{lj} - \hat{y}_{ls} - 1)^2$$

$$\begin{cases} \hat{r}_{ij} = \mathbf{w}^T \mathbf{e}_h, \ if \ \mathbf{e}_0 = \varphi_{pooling}(\mathbf{u}_i, \mathbf{v}_j) \\ \hat{y}_{lj} = \mathbf{w}^T \mathbf{e}_h, \ if \ \mathbf{e}_0 = \varphi_{pooling}(\mathbf{g}_l(j), \mathbf{v}_j) \end{cases}$$




群体中各用户作用不同




$$o(j,t) = \mathbf{h}^T \mathrm{ReLU}(\mathbf{P}_v \mathbf{v}_j + \mathbf{P}_u \mathbf{u}_t + \mathbf{b})$$

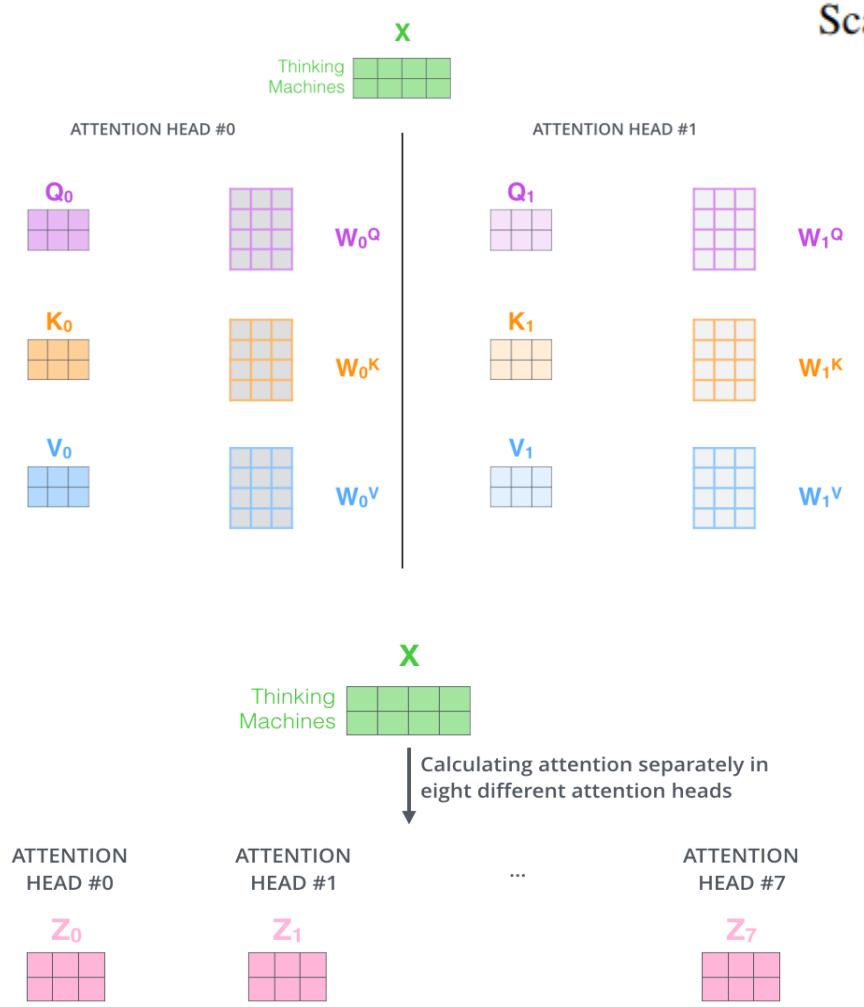
$$\alpha(j,t) = \mathrm{softmax}(o(j,t)) = \frac{\exp o(j,t)}{\sum_{t' \in \mathcal{K}_l} \exp o(j,t')}$$

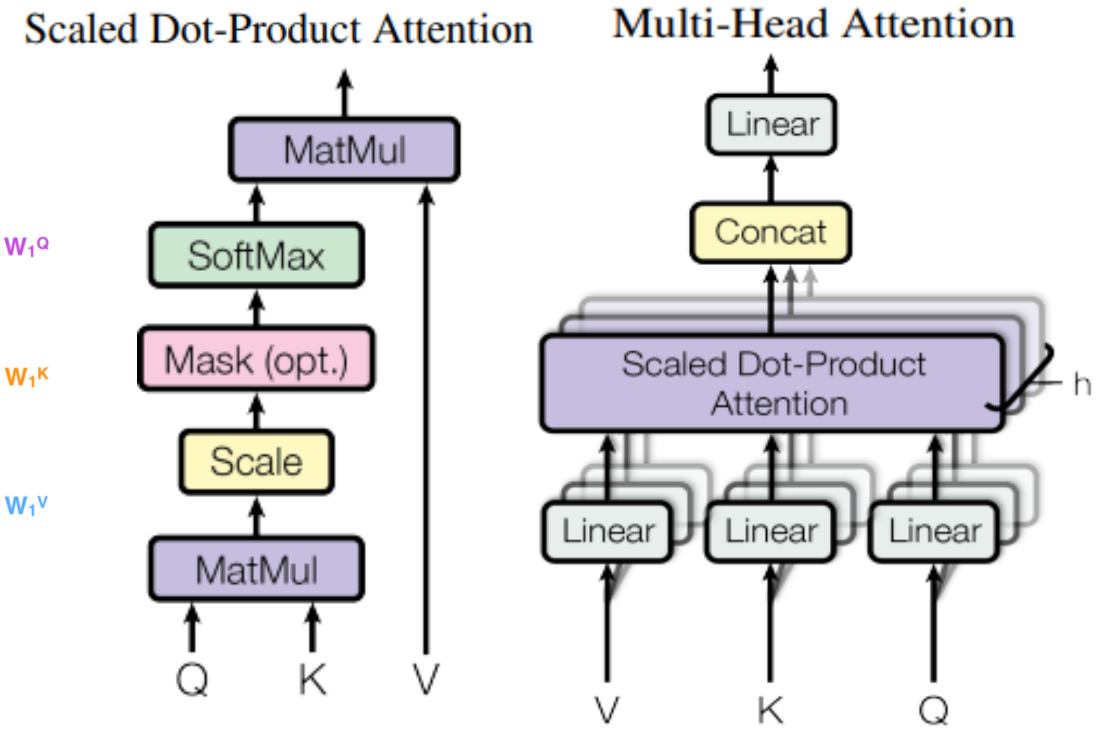
$$\mathbf{g}_l(j) = \underbrace{\sum_{t\in\mathcal{K}_l} \alpha(j,t)\mathbf{u}_t}_{\text{user embedding aggregation}} + \underbrace{\mathbf{q}_l}_{\text{group preference embedding}}$$

j: item

t: user

# Multi-head attention



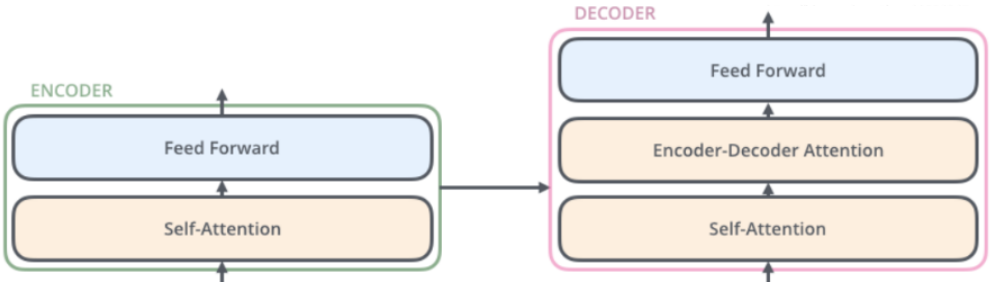

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

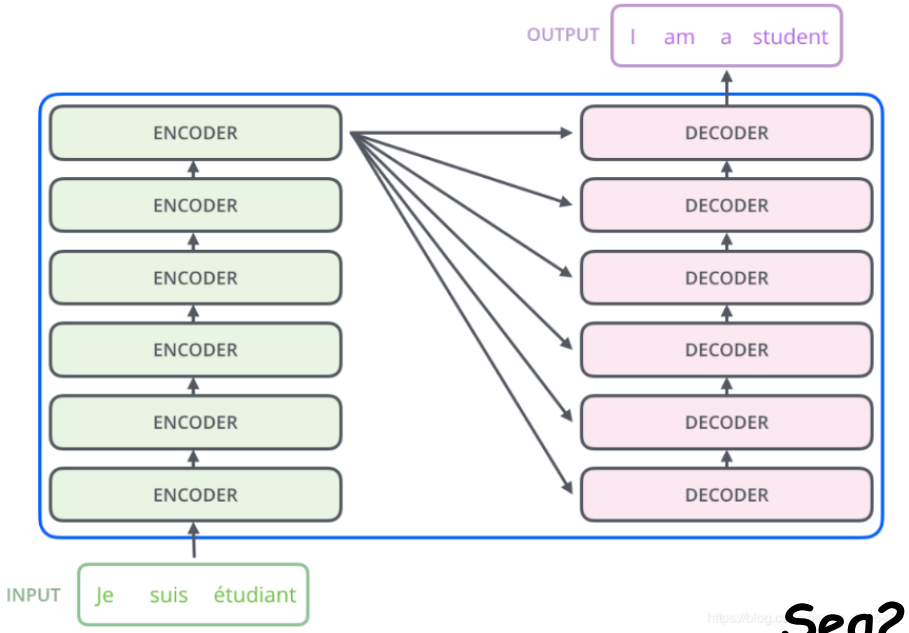
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

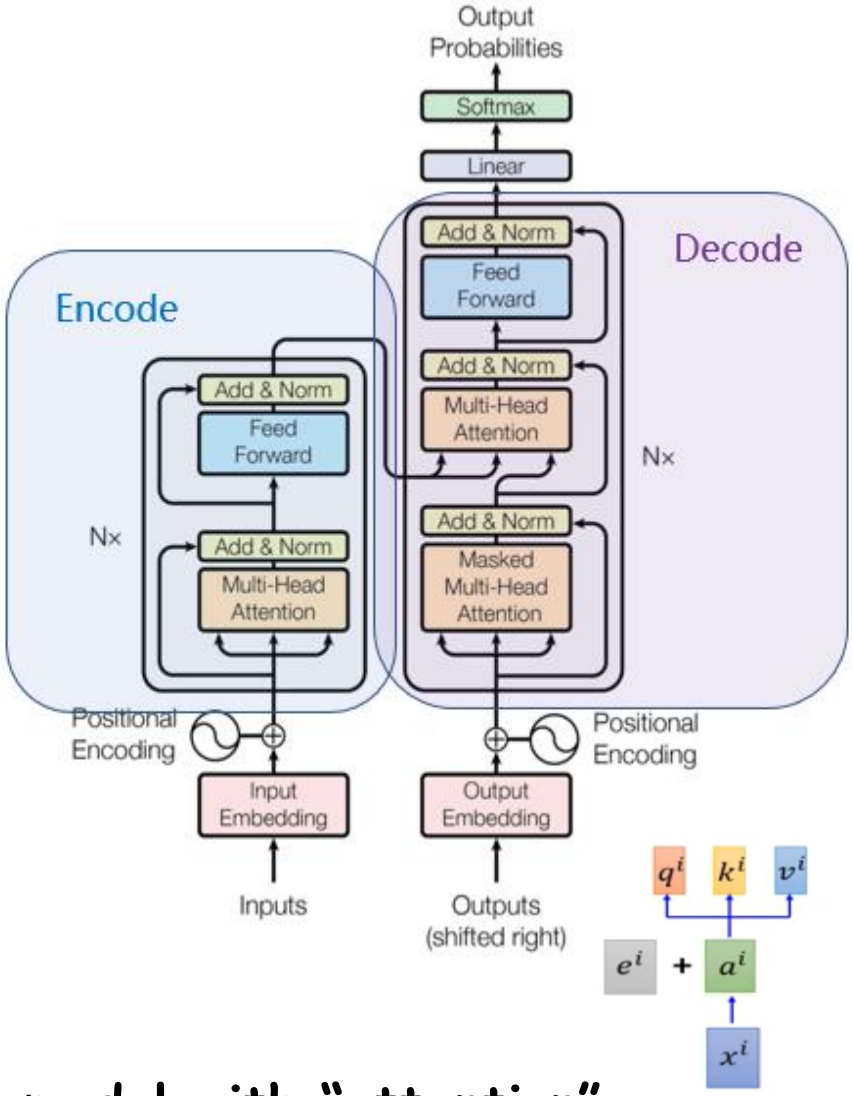
$$\text{head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

# Transformer



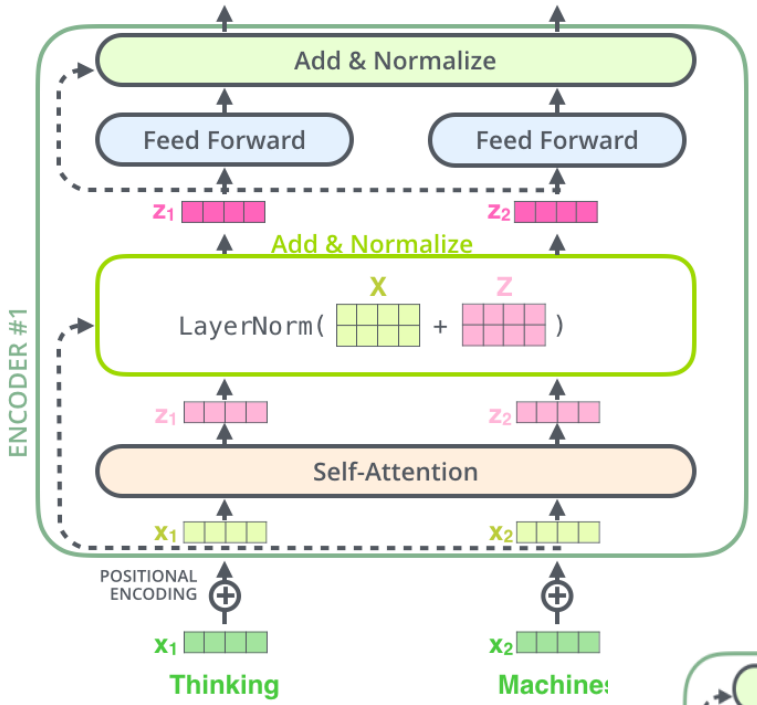



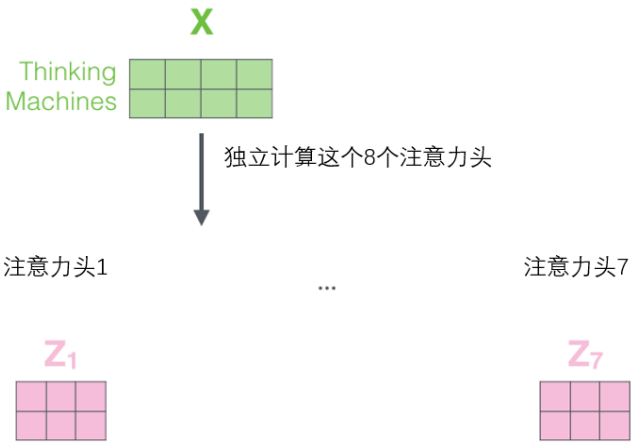



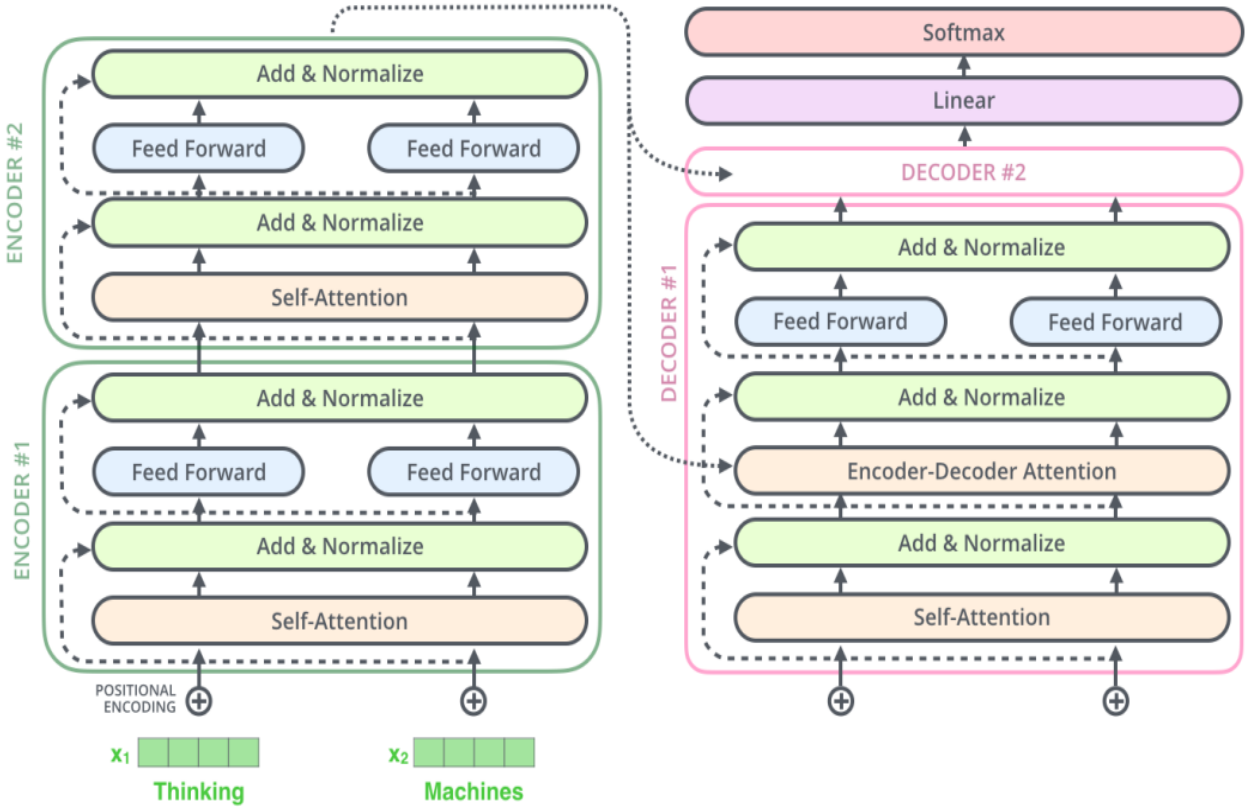




**Seq2seq model with "attention"**

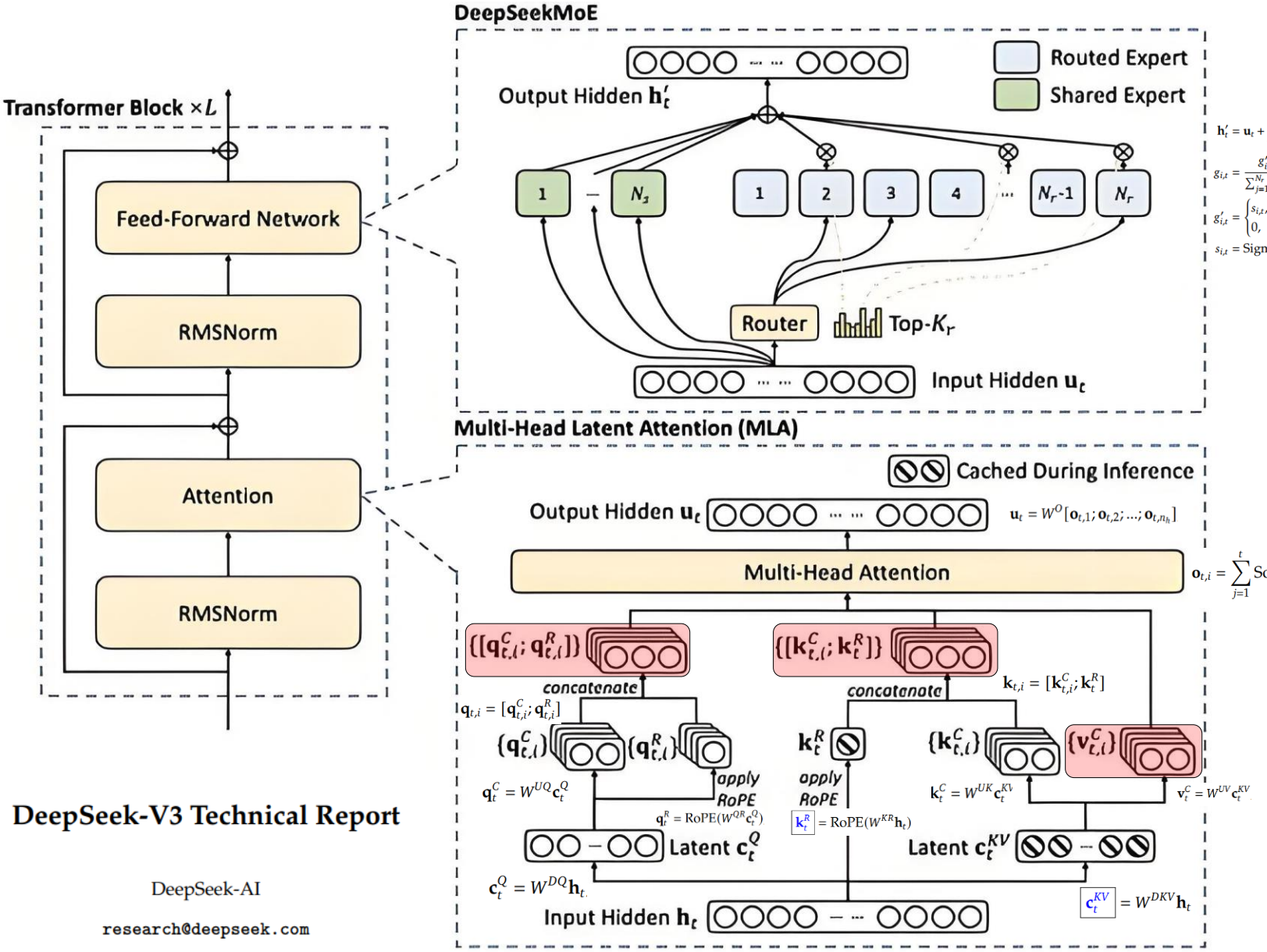
Attention Is All You Need (NIPS217)


Add & Normalize
Feed Forward
Feed Forward
z1
z2
Add & Normalize
X
Z
LayerNorm( + )
ENCODER #1
z1
z2
Self-Attention
x1
x2
POSITIONAL ENCODING
x1
x2
Thinking
Machine


X
Thinking
Machines
独立计算这个8个注意力头
注意力头0
注意力头1
...
注意力头7
Z0
Z1
Z7


Softmax
Linear
DECODER #2
DECODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Encoder-Decoder Attention
Add & Normalize
Self-Attention
ENCODER #2
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
ENCODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
POSITIONAL ENCODING
x1
x2
Thinking
Machines

# Deepseek

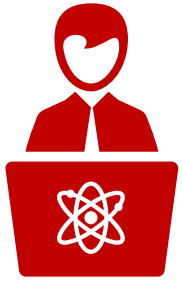



**DeepSeek-V3 Technical Report**


DeepSeek-AI

research@deepseek.com

中国人民大学
RENMIN UNIVERSITY OF CHINA



# 谢谢大家！

